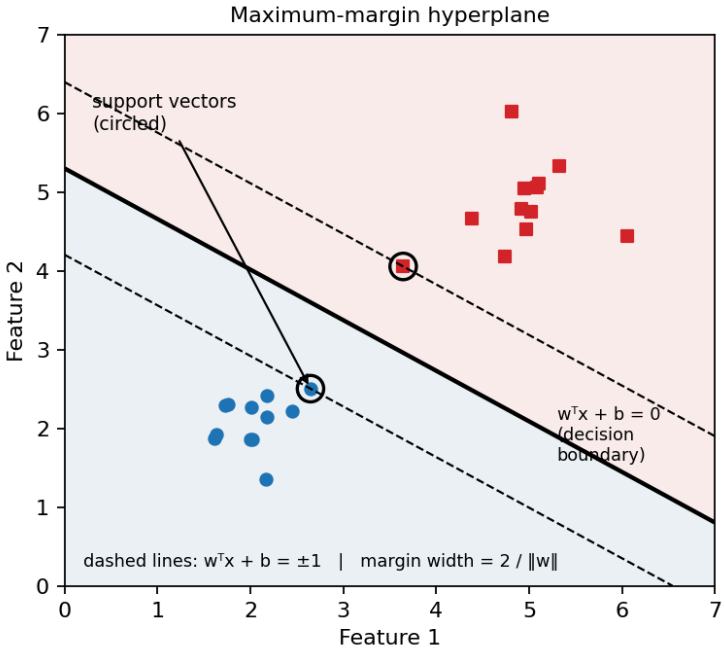
Support Vector Machines (SVM) සහ Support Vector Regression (SVR)

Classification, regression සහ kernels + Neural Networks හැඳින්වීම

දේශන සටහන් – 7 වන දිනය (13 වන දා) • සරල සිංහලෙන් • R code, සැබෑ R outputs, plots සහ විසඳූ ගැටලු සමඟ



මෙහි අඩංගු දේ

කොටස	මාතෘකාව
0	පන්තියේ කරුණු (කළ දේ, homework, lab, group project)
1	පුනරීක්ෂණය: supervised learning, train/test split, cross-validation, "ඉරි අඳින" වෙනත් ක්‍රම
2	Hyperplanes සහ linear classifiers
3	හොඳම ඉර කුමක්ද? Maximum-margin අදහස සහ support vectors
4	Hard-margin SVM (ගණිතය, සරලව)
5	Noisy දත්ත, soft margin, slack variables, cost C, hinge loss
6	Dual problem (support vectors වැදගත් ඇයි)
7	Multi-class SVM: one-vs-one උපක්‍රමය
8	Non-linear SVM සහ kernel trick (linear, polynomial, RBF, sigmoid)
9	Support Vector Regression (ε-SVR) සහ ε-tube
10	දත්ත සකස් කිරීම, leakage නැතිව tuning, ඇගයීමේ මිනුම්
11	R demo: SVM classification (linear, RBF, tuning, probabilities, class weights)
12	R demo: Support Vector Regression
13	පන්තියේ ක්‍රියාකාරකම් (iris versicolor vs virginica; mtcars lm vs SVR) + homework
14	සුලබ වැරදි සහ වෙනත් ක්‍රම සමඟ සැසඳීම
15	Exercises 1–7 විසඳුම් සමඟ
16	Neural Networks හැඳින්වීම
17	ප්‍රධාන කරුණු සහ R cheat-sheet

0. පන්තියේ කරුණු

- පසුගිය පන්තිය (6 වන දා): **recommendation systems** + exercises සහ පන්ති ක්‍රියාකාරකමක්. සියලු files LMS එකට upload කර ඇත.
- අද: නව algorithm එකක් – classification සඳහා **Support Vector Machine (SVM)** සහ regression සඳහා **Support Vector Regression (SVR)**. ඉන්පසු **Neural Networks** පටන් ගැනීම.
- Syllabus එකට අනුව SVM සහ kernels පෙර පාඨමාලාවේ ආවරණය කර නැති නිසා මෙහි ඇතුළත් කළා. SVM වලින් පසු machine-learning කොටස අවසන්; ඉතිරි වන්නේ **neural networks සහ deep learning**.
- **Group project:** R සහ Python දෙකම ආවරණය වීමට **Python** භාවිතා කරන්න කියන්න දේශකතුමා සැලසුම් කරනවා. උපදෙස් LMS එකේ එනු ඇත.
- **Assignment (2 වන කොටස):** මූලික regression models භාවිතා කළ පළමු Kaggle assignment එක මතකද? Advanced models (SVR වැනි) ඉගෙන ගත් පසු, ප්‍රතිඵල වැඩි දියුණු කිරීමට ඒ ගැටලුවටම නැවත යනවා.
- **Homework:** iris ක්‍රියාකාරකම සඳහා මුලින්ම දත්ත split කර, ඉන්පසු SVM, **logistic regression** සහ **random forest** සමඟ සසඳන්න. Upload link එක LMS එකේ ඇත.
- **Lab sheet** එක LMS එකේ දමනු ඇත – **සඳුදා** එය කරන්න. Lab sessions වලට නිතිපතා සහභාගී වන්න.
- දේශකතුමාගේ පණිවිඩය: "පළමු දවසේ සිටම computer එකක් ඉදිරියේ ඉඳගෙන ඉන්න ඕන. Code කරන්නේ නැත්නම් ඉගෙනීමක් නැහැ. Marks ආවට පස්සේ මට email කරලා ඉල්ලන්න එපා – පුහුණු වෙලා ඔබේ grade එක ඔබම ලබා ගන්න ඕන."

Interview ඉඟිය

"SVM ක්‍රියා කරන්නේ classes දෙකකට – classes ගොඩක් තියෙනකොට ඒක භාවිතා කරන්නේ කොහොමද?" වැනි ප්‍රශ්න **technical interviews** වල බහුලයි. මෑතකදී internships සඳහා ගිය data-science සිසුන්ට technical interviews දෙකක් තිබුණා, "inside-out" අහලා තියෙනවා. Code එක run කරන විදිය පමණක් නොවෙයි, **ඇතුළේ මොකද වෙන්නේ** කියලත් දැනගන්න ඕන.

1. පුනරීක්ෂණය: supervised learning සහ "ඉරි ඇඳීම"

1.1 Supervised learning

SVM සහ SVR **supervised learning** ක්‍රම වේ: **label** එක (target / response variable) තියෙනවා, ඒ නිසා model එකට "අධීක්ෂණය යටතේ" ඉගෙන ගන්න පුළුවන්.

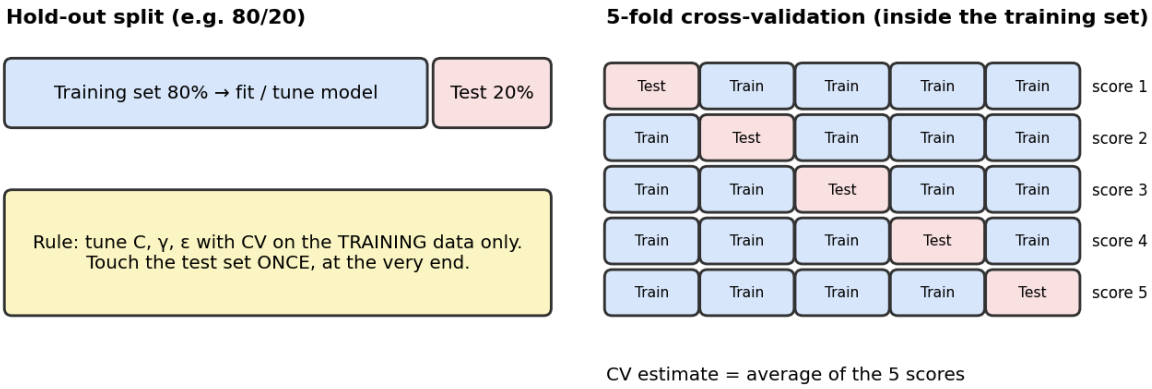
කාර්යය	Response (label)	මෙම දේශනයේ ක්‍රමය
Classification	Categorical (උදා: cat / mouse, setosa / versicolor)	**SVM** – Support Vector Machine
Regression	Quantitative / continuous (උදා: mpg, marks)	**SVR** – Support Vector Regression

1.2 Training set, test set සහ cross-validation

Machine learning වලදී අපි දත්ත කොටස් දෙකකට බෙදනවා:

- **Training set** (උදා: අහඹු ලෙස 70% හෝ 80%) – model එක fit (train) කිරීමට.
- **Test set** (ඉතිරි 30% හෝ 20%) – අවසානයේ predictions ගැනීමට. **Predicted** අගයන් **observed** අගයන් සමඟ සසඳා model එක හොඳද කියා බලනවා.

ඉරක්/වක්‍රයක් ඇත්තටම වැඩ කරනවාද බැලීමට (සහ models අතරින් තෝරා ගැනීමට) **hold-out validation** හෝ **k-fold cross-validation** අවශ්‍යයි. වෙනම validation/test set එකක් නැත්නම් අපි දත්ත **overfit** කරනවා.



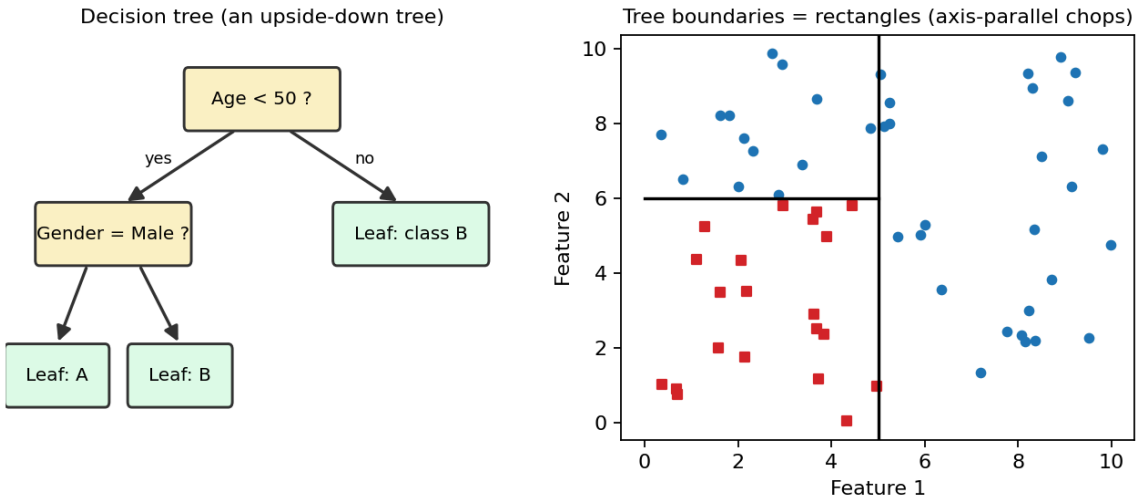
Hold-out split සහ 5-fold cross-validation

k-fold cross-validation (උදා: k = 5): දත්ත කොටස් 5 කට කඩනවා. 1 වන වටයේදී 1 වන කොටස test කොටස, අනෙක් 4 training සඳහා; 2 වන වටයේ 2 වන කොටස test කොටස, ආදී ලෙස. මෙය 5 වතාවක් කර ලකුණුවල **සාමාන්‍යය** ගන්නවා. ක්‍රම දෙකම වැඩ කරනවා, නමුත් k-fold CV **වඩා ශක්තිමත් (robust)** ("පහසු උදාහරණ" සහිත එක වාසනාවන්ත split එකක් මත රඳා පවතින්නේ නැහැ).

1.3 Predictive models සාදන වෙනත් ක්‍රම ("ඉරි ඇඳීම")

සෑම classifier එකක්ම ඇත්තටම classes වෙන් කරන ඉරක් (හෝ වක්‍රයක්) අඳින ක්‍රමයකි. අපි දැක ඇති ඒවා:

- **Decision trees** – උඩු යටිකුරු කළ ගසක් ලෙස සිතන්න. එක් එක් අත්තේ කොන්දේසියක් ඇත (උදා: *Age < 50?* ඉන්පසු *Gender = Male?*). ගස feature space එක "කපනවා"; කැපීම් error අවම කරන feature splits වේ. මායිම් **සෘජුකෝණාස්‍ර (rectangles)**.
- **k-nearest neighbours (KNN)** – training දත්තවල ළඟම ඇති k points බලා බහුතර class එක ගැනීම.
- **Random forests** – decision trees ගොඩක් එකට (කලින් ආවරණය කළා).



Decision tree එකක් සහ එය සාදන සෘජුකෝණාස්‍රාකාර decision boundaries

"ඉරි ඇඳීමේ තරගය" (slides වලින්): එකම දත්ත ක්‍රම කිහිපයකට යවනවා, එක් එක් ක්‍රමය එහි hyper-parameter grid එකක් මත tune කරනවා (KNN: k = 5 සිට 10; tree: cp = 0.01 සිට 0.10; SVM: C = 0.001 සිට 10). හොඳම 5-fold CV accuracy ඇති එක තෝරනවා (slide උදාහරණයේ: KNN හොඳම k = 3, 95%; tree හොඳම cp = 0.02, 98%; SVM හොඳම C = 9, 100%).

ඉරි මෙතරම් වැදගත් ඇයි? සැබෑ රටාව අපි දන්නේ නැහැ; ඇඳිය හැකි ඉරි අනන්ත ගණනක් ඇත; features බොහොමයක් ඇති විට අතින් අඳින්න බැහැ; බුද්ධිමත් හැසිරීම අනුකරණය කිරීමට ඉතා සංකීර්ණ ඉරි අවශ්‍යයි.

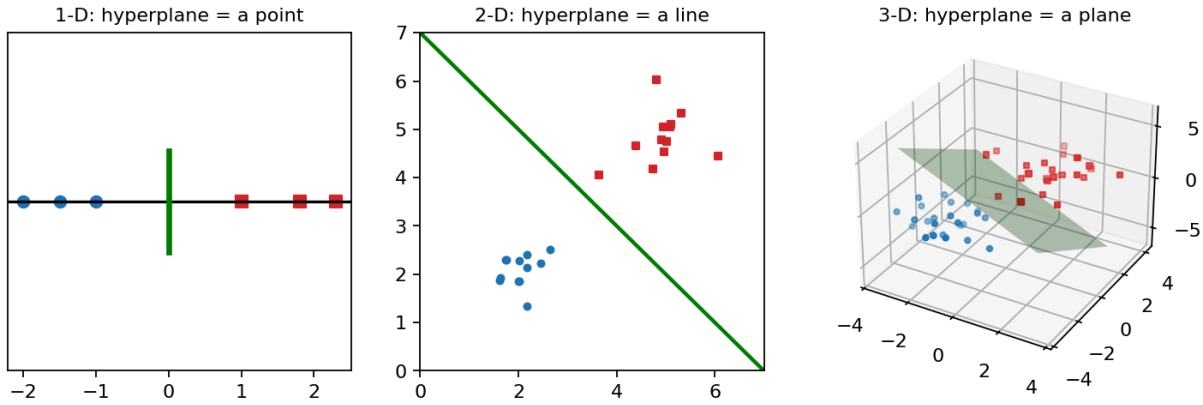
1.4 ටිකක් ඉතිහාසය (සම්පූර්ණත්වය සඳහා)

- **Deep Blue (1997)** – IBM computer එක ලෝක chess ශූරයා Garry Kasparov පරාජය කළා. එය data mining භාවිතා කළේ නැහැ, නමුත් තවමත් සිත්ගන්නා සුළුයි.
- **AlphaGo** – **Go** පුවරු ක්‍රීඩාව සඳහා Google හි program එක (chess වගේ, නමුත් හැකි චලන ඉතා වැඩියි). එය ඉහළම මිනිස් ක්‍රීඩකයෙකු පරාජය කළා – විශාල පුවතක්. AlphaGo යනු ඊළඟ චලනය පුරෝකථනය කළ ඉතා සංකීර්ණ predictive model එකක් පමණයි. එයට පියවර ගණනාවක් ඉදිරියට සිතා, සියලු හැකියාවන් ඇගයීමට ලක් කර optimise කරන්න වෙනවා (තමන්ගේ කැට රැකගෙන, විරුද්ධවාදියාගේ කැට ලබා ගනිමින්). එය සෑදීමට supercomputer එකක මාස ගණනක් ගත වුණා.

මේ දක්වා දුටු data visualisation ක්‍රම: scatter plots (තලයක points), line plots (සංඛ්‍යාත්මක vs සංඛ්‍යාත්මක, උදා: time series), bar graphs (categorical vs සංඛ්‍යාත්මක), histograms (continuous variable එකක ව්‍යාප්තිය), heat maps (categorical vs categorical).

2. Hyperplanes සහ linear classifiers

SVM හි අදහස: classes දෙකක් වෙන් කරන ඉරක් ඇඳිය හැකි නම්, නව points පහසුවෙන් classify කළ හැක. සාමාන්‍යයෙන් මෙම වෙන් කරන වස්තුව **hyperplane** ලෙස හැඳින්වේ:



මාන 1 දී hyperplane එකක් point එකකි, මාන 2 දී ඉරකි, මාන 3 දී තලයකි (plane – පැතලි මතුපිටක්, ගුවන් යානයක් නොවේ!)

Observation එකකට predictor vector එකක් ඇත: $x = (x_1, x_2, \ldots, x_p)^T$. p-මාන අවකාශයක hyperplane එකක් යනු

$$\mathbf{w^T x + b = 0}$$

මෙහි $w = (w_1, \ldots, w_p)^T$ hyperplane එකට **normal** (ලම්බක) වන අතර b යනු intercept එකකි. ($y = mx + c$ ඉරක් සමඟ සසඳන්න: අපට **slope** එකක් සහ **intercept** එකක් අවශ්‍යයි.)

Binary classification සඳහා classes $y_i \in \{-1, +1\}$ ලෙස code කරනවා. තීරණ රීතිය:

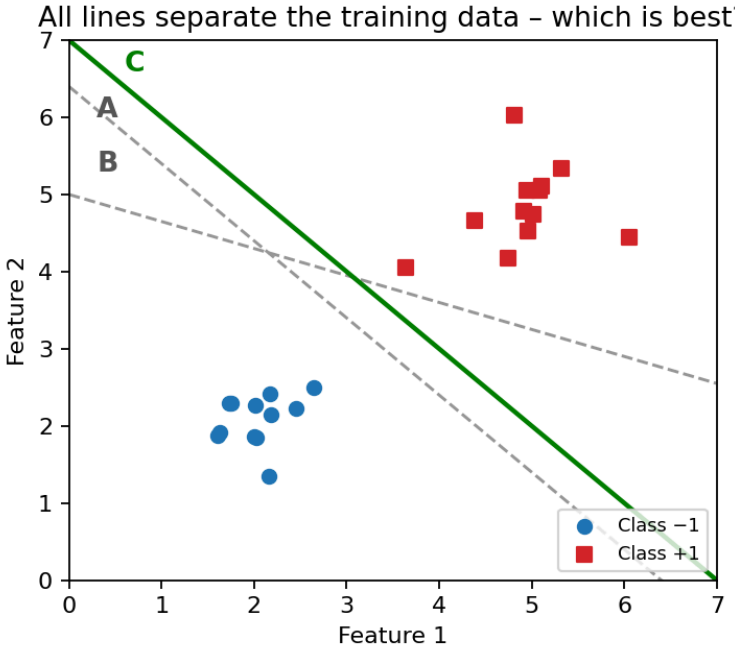
$$\hat{y}(x) = +1 \text{ if } w^T x + b > 0, \quad \hat{y}(x) = -1 \text{ if } w^T x + b < 0$$

$|w^T x + b|$ හි විශාලත්වය x, hyperplane එකේ සිට කොතරම් දුරින්ද යන්නට සම්බන්ධයි, නමුත් එය ලම්බක දුර **නොවේ**. **ලකුණ සහිත ලම්බක දුර (signed perpendicular distance)** මෙයයි:

$$(w^T x + b) / \|w\|$$

3. හොඳම ඉර කුමක්ද? Maximum-margin අදහස

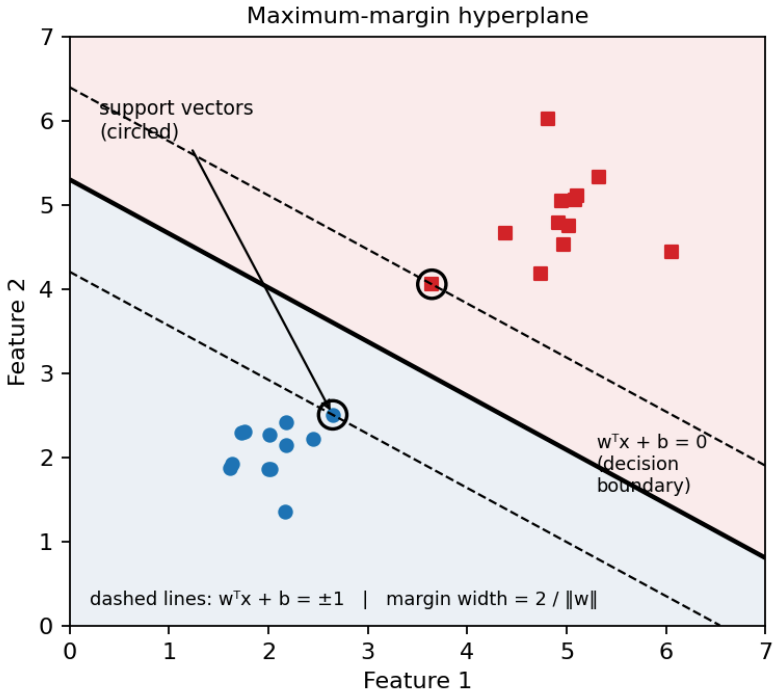
පන්ති ප්‍රශ්නය (MCQ ආකාරයේ, වාර අවසාන විභාගයේ වගේ): පහත A, B, C ඉරි තුනම training දත්ත නිවැරදිව වෙන් කරයි. හොඳම එක කුමක්ද?



පිළිතුර: මැද ඇති ඉර (C). එක් class එකකට ඉතා ආසන්න ඉරක් training දත්ත මත වැඩ කරයි, නමුත් එය **ශක්තිමත් (robust) නැහැ**. නව test points එන විට, පැරණි ඒවාට වඩා ටිකක් ඔබ්බෙන් ඇති point එකක් වැරදි පැත්තට වැටෙන්න පුළුවන්. හරියටම මැද ඇති ඉර දෙපැත්තටම වැඩිම ඉඩ තබන නිසා, වැඩි නව points ප්‍රමාණයක් නිවැරදිව classify කරයි.

3.1 SVM හි පියවර දෙක

1. **"පිටත" points හඳුනා ගැනීම** – එක් එක් class එකේ අනෙක් class එකට ආසන්නම points. මේවා **support vectors** ලෙස හැඳින්වේ.
2. **සමදුර (equidistant) ඉරක් ඇඳීම** – support vectors කට්ටල දෙකට එකම දුරින්, classes අතර **උපරිම margin** (විශාලතම හිඩැස) ඇති ඉර.



Support vectors (රවුම් කළ) ඉර තීරණය කරයි. Solid ඉර = decision boundary; dashed ඉරි = margin මායිම්.

- මෙය **maximum-margin hyperplane** ලෙස හැඳින්වේ.
- **Support vector නොවන** point එකක් ටිකක් චලනය කිරීමෙන් සාමාන්‍යයෙන් ඉර වෙනස් නොවේ; **support vector** එකක් චලනය කළොත් ඉර වෙනස් විය හැක.
- මෙම මැද ඉර සොයා ගැනීමට නිවැරදි **slope සහ intercept** අවශ්‍යයි – support vectors අතර **margin එක උපරිම කිරීමේ** optimisation කාර්යයකි. අදහස පහසුයි, නමුත් පිටුපස ලස්සන ගණිතයක් ඇත (ඊළඟ කොටස).

> **අවසාන විභාගය ගැන සටහන (slides වලින්)**
> "SVM මෙම ඉර අඳින්නේ **කෙසේද/ඇයි** යන්න ගැන ප්‍රශ්නයක් අහනවා." පිළිතුර: එය support vectors සොයා, ඒවාට සමදුරින් විශාලතම margin එක සහිත ඉර අඳියි, මොකද විශාල margin එකක් නොදුටු දත්තවලට වඩා හොඳින් generalise වන ස්ථාවර මායිමක් ලබා දෙයි.

4. Hard-margin SVM (ගණිතය, සරලව)

මෙය applied පාඨමාලාවක් නිසා ගණිතය ව්‍යුත්පන්න කරන්න අවශ්‍ය නැහැ, නමුත් මොකද වෙන්නේ කියා දැනගන්න ඕන. (MIT/Harvard පාඨමාලා පටන් ගන්නේ හරියටම මෙතනින්: "hyperplane එකේ සමීකරණය $w^T x + b = 0$".)

දත්ත **සම්පූර්ණයෙන්ම linear ලෙස වෙන් කළ හැකි** යැයි උපකල්පනය කරමු. w සහ b එකම ධන සංඛ්‍යාවකින් ගුණ කළත් එකම ඉර ලැබෙන නිසා, ළඟම points සඳහා $y_i(w^T x_i + b) = 1$ වන ලෙස scale එක තෝරනවා. එවිට සෑම point එකක්ම මෙය තෘප්ත කළ යුතුයි:

$$y_i(w^T x_i + b) \geq 1$$

Margin ඉරි දෙක $w^T x + b = 1$ සහ $w^T x + b = -1$. ඒවා අතර දුර (margin පළල):

$$\mathbf{2 / \|w\|}$$

එබැවින් margin එක උපරිම කිරීම = $\|w\|$ අවම කිරීම = (පහසුවට) $\frac{1}{2}\|w\|^2$ අවම කිරීම. Hard-margin ගැටලුව:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 \text{ subject to } y_i(w^T x_i + b) \geq 1,\ i = 1, \ldots, n$$

මෙය **convex quadratic** optimisation ගැටලුවකි, එබැවින් ඕනෑම local optimum එකක් global optimum එකද වේ (හිරවීමේ අවදානමක් නැහැ).

Functional margin සහ geometric margin

නම	සූත්‍රය	සටහන
Functional margin	$y_i(w^T x_i + b)$	w සහ b හි අත්තනෝමතික scale එක මත රඳා පවතී
Geometric margin	$y_i(w^T x_i + b) \,/\, \|w\|$	සැබෑ දුර – scale එක මත රඳා නොපවතී

Hard margin හි සීමාව

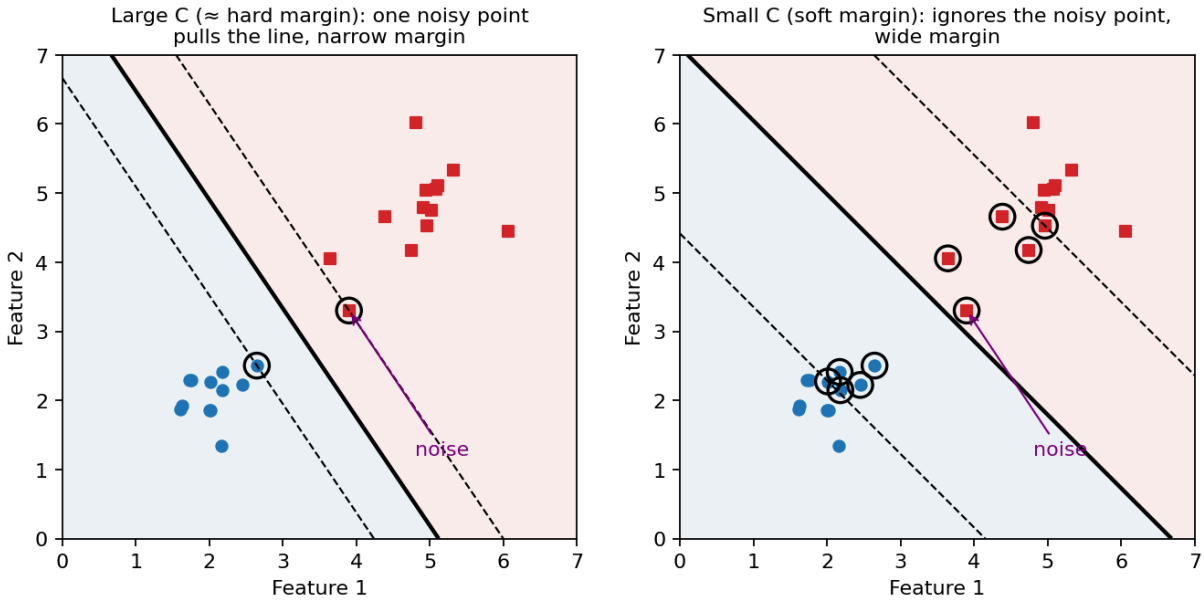
එක observation එකක් හෝ සම්පූර්ණ වෙන්වීම වළක්වයි නම්, සියලු constraints තෘප්ත කළ නොහැක (infeasible). සැබෑ දත්තවල overlap, noise සහ outliers ඇත. මෙය **soft margin** එකට මග පාදයි.

5. Noisy දත්ත, soft margin සහ cost C

5.1 Noisy observations ඉර විනාශ කළ හැක

Slides උදාහරණය: බළලුන් සහ මීයන්. මීයෙකු වගේ පෙනෙන **විකෘති බළලෙක්** සිටියොත්? මෙම noisy point එක **support vector** එකක් ලෙස තෝරා ගත්තොත්, ඉර එය දෙසට යයි.

- **තනි noisy උදාහරණයක්** සියල්ල අවුල් කරයි.
- Support vector එක වැරදියට තෝරා ගැනේ.
- Model එක අවලංගුයි, **overfit** වී ඇත – සැබෑ ලෝක අවස්ථාවලට හොඳින් generalise නොවේ.



එක් noisy රතු point එකක් සහිත එකම දත්ත. වම: විශාල C noise එක පසුපස යයි. දකුණ: කුඩා C එය නොසලකා පළල් margin එකක් තබා ගනී.

5.2 Slack variables

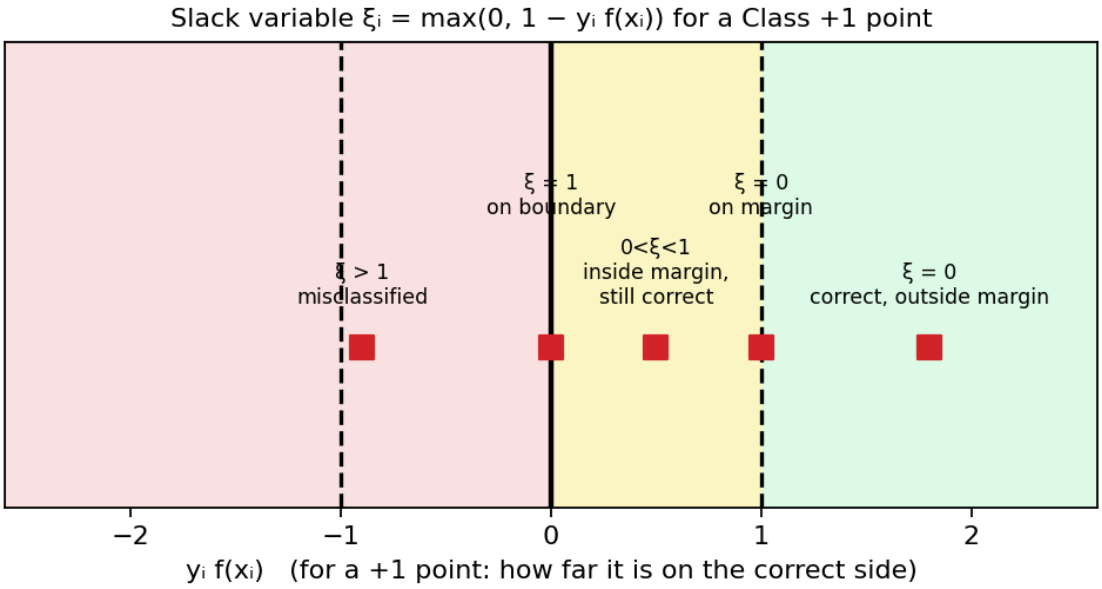
වැරදි කිහිපයකට ඉඩ දීමට, සෑම observation එකකටම සෘණ නොවන **slack variable** එකක් ξ_i ("xi") දෙනවා:

$$y_i(w^T x_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0$$

Soft-margin ගැටලුව:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + \mathbf{C}\ \Sigma\ \xi_i \text{ (ඉහත constraints යටතේ)}$$

$C > 0$ යනු **cost** (regularisation) parameter එකයි.



Class +1 point එකක් පිහිටන තැන සහ එහි slack අගය

ξ_i අගය	තේරුම
$\xi_i = 0$	Margin එකේ නිවැරදි පැත්තේ (margin ඉර මතම විය හැක)
$0 < \xi_i < 1$	නිවැරදිව classify වී ඇත, නමුත් margin එක **ඇතුළත**
$\xi_i = 1$	හරියටම decision boundary එක මත
$\xi_i > 1$	**වැරදියට classify වී ඇත**

5.3 Cost parameter C

KNN හි **k**, decision trees හි **cp** සහ **max depth** මෙන්, SVM වලටද model සංකීර්ණත්වය පාලනය කරන **hyper-parameter** එකක් ඇත – එනම් noisy observations වලට කොතරම් ඉවසනවාද. මෙය **cost C**: support vectors තෝරන විට training point එකක් වැරදියට classify කිරීමේ "වියදම". එය තෝරාගත් support vectors noise වලට කොතරම් ප්‍රතිරෝධීද යන්න තීරණය කරයි.

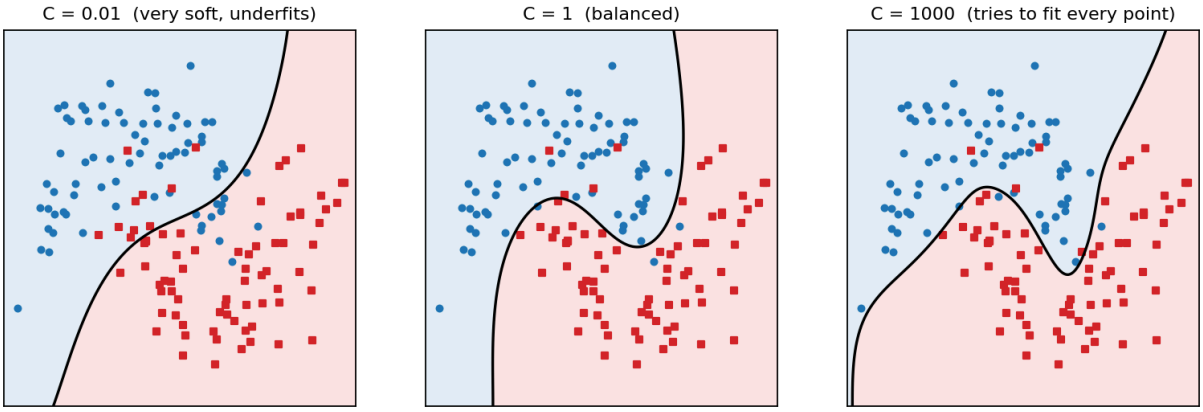
- **විශාල C** (උදා: C = 1,000): training දත්තවල වැරදි ඉතා මිලයි → "අපට වැරදි කරන්න බැහැ" → **පටු margin**, සෑම training point එකක්ම නිවැරදිව classify කිරීමට උත්සාහ කරයි. Outliers වලට වඩා සංවේදී → ඉහළ variance → **overfitting** අවදානම.
- **කුඩා C** (උදා: C = 0.001): වැරදි ලාභයි → "අපට වැරදි කිහිපයක් කරන්න පුළුවන්" → **පළල් margin**, වඩා regularised. Training error වැඩි විය හැකි නමුත් බොහෝ විට වඩා හොඳින් generalise වේ. ඉතා කුඩා නම් → **underfitting**.
- "Cost" හි සැබෑ ජීවිත අර්ථය: ඩෙංගු (හෝ පිළිකා) නැති රෝගියෙකුට ඔහුට එය ඇතැයි කීම ඉතා මිල අධික වැරැද්දකි.
- C මගින් **bias–variance trade-off** එකක් පාලනය කරයි. අපි C අගයන් ගොඩක් (cross-validation භාවිතයෙන්) උත්සාහ කර overfitting සහ underfitting දෙකම වළක්වන එක තෝරනවා.

ප්‍රවේශම් – slides හි එක් ප්‍රකාශයක් ප්‍රතිවිරුද්ධ පැත්තට ඇත

Slides (සහ පන්තියේ කියවීම) කියන්නේ: "Higher C … ඉතා ඉහළ නම් *underfit* වේ. Lower C … ඉතා අඩු නම් *overfit* වේ." සම්මත SVM න්‍යායට අනුව එය **ප්‍රතිවිරුද්ධයි**:

- **C ඉතා ඉහළ → overfit** (පටු margin, noise පසුපස යයි, ඉහත වම් plot එක වගේ).
- **C ඉතා අඩු → underfit** (ඉතා පළල් margin, උල්ලංඝන ගොඩකට ඉඩ දෙයි).

පාඨමාලාවේ ලිඛිත සටහන (4.2 කොටස) සහ Exercise 3 මෙම සම්මත අනුවාදයට එකඟයි – විභාගයේදී මෙය භාවිතා කරන්න.



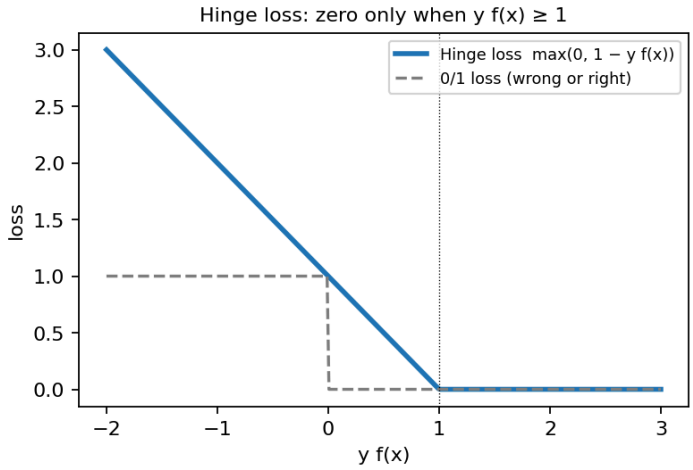
RBF kernel එකක් සමඟ C හි බලපෑම (γ ස්ථාවරයි): කුඩා C ඉතා සරලයි; ඉතා විශාල C තනි points වටා නැමෙයි

5.4 Hinge loss

Soft-margin ගැටලුව **hinge loss** $L_i = \max\{0,\ 1 - y_i\ f(x_i)\}$ මගින්ද ලිවිය හැක ($f(x) = w^T x + b$):

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \sum \max\{0,\ 1 - y_i\ f(x_i)\}$$

පළමු පදය **පළල් margin** එකකට කැමතියි; දෙවන පදය තම margin ඉරේ වැරදි පැත්තේ හෝ ඉර මත ඇති points වලට **දඬුවම් (penalty)** කරයි.



6. Dual problem සහ support vectors

Lagrange multipliers α_i භාවිතයෙන්, soft-margin SVM හට **dual ආකාරයක්** ඇත:

$$\text{maximise } \sum \alpha_i - \tfrac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j\ \mathbf{x_i}^T \mathbf{x_j} \text{ subject to } 0 \le \alpha_i \le C,\ \sum \alpha_i y_i = 0$$

Fit කළ coefficients: $w = \Sigma\ \alpha_i\ y_i\ x_i$. **$\alpha_i > 0$ ඇති observations පමණක් දායක වේ – මේවා support vectors.** Decision function එක:

$$f(x) = \Sigma_{i \in S}\ \alpha_i\ y_i\ x_i^T x + b \text{ (S = support vectors කට්ටලය)}$$

මෙය වැදගත් ඇයි

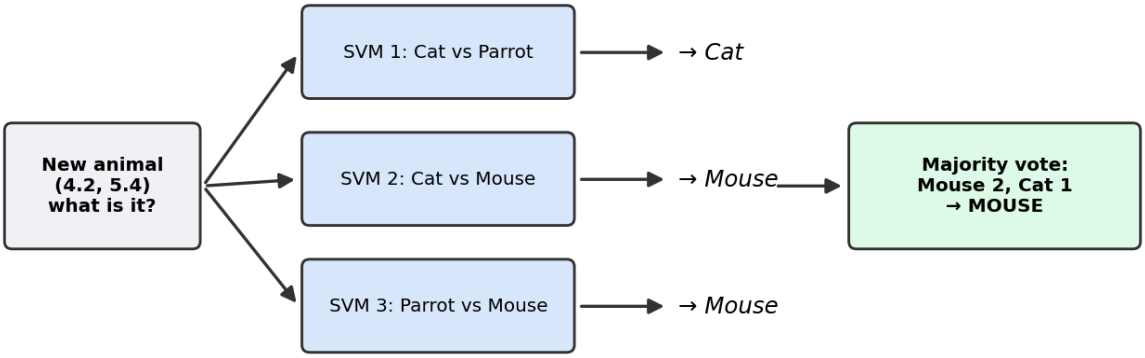
දත්ත පෙනී සිටින්නේ **inner products** $x_i^T x_j$ හරහා පමණි. මෙම inner products **kernel** $K(x_i, x_j)$ එකකින් ප්‍රතිස්ථාපනය කළොත්, non-linear මායිම් නොමිලේ ලැබේ. මෙය kernel trick එකට දොරටුවයි (8 කොටස).

7. Multi-class SVM: one-vs-one උපක්‍රමය

Classes දෙකකට වඩා තිබේ නම් – උදා: **බළලුන්, මීයන් සහ ගිරවුන්**? දැන් ඉර අඳින්නේ කොහොමද? KNN සහ decision trees classes ගොඩක් ස්වභාවිකවම හසුරුවයි. නමුත් SVM වෙන් කරන්නේ classes **දෙකක්** පමණයි, එබැවින් එය තනියම භාවිතා කළ නොහැක. අපි උපක්‍රමයක් භාවිතා කරනවා.

7.1 One-vs-one (OvO)

1. **සෑම classes යුගලයකටම** එක් SVM එකක් train කිරීම: Cat vs Parrot, Cat vs Mouse, Mouse vs Parrot (SVM තුන් වතාවක් run කිරීම → **sub-models** තුනක්).
2. නව (නොදන්නා) observation එකක් එන විට එය **එක් එක්** classifier එකෙන් ඇගයීම.
3. අවසාන prediction එක ලෙස **බහුතර ඡන්දය (majority vote)** ගැනීම.



Slides වලින් විසඳූ උදාහරණය: (4.2, 5.4) හි ඇති නව සත්වයා

විසඳූ උදාහරණය: නව point එක (4.2, 5.4). Cat vs Parrot → **Cat**; Cat vs Mouse → **Mouse**; Parrot vs Mouse → **Mouse**. ඡන්ද: Mouse 2, Cat 1 → prediction = **Mouse**.

- K classes සඳහා OvO ට binary classifiers **K(K − 1)/2** ක් අවශ්‍යයි (K = 3 → 3; K = 4 → 6).
- **One-vs-rest (OvR)** තවත් විකල්පයකි: එක් class එකකට එක් classifier එකක් (එම class එක vs අනෙක් සියල්ල).
- "මෙය ටිකක් වංචාවක් වගේ, නමුත් SVM වලට තනියම multi-class හසුරුවන්න බැහැ."
- **R මෙම උපක්‍රමය පසුබිමේ අප වෙනුවෙන් කරයි:** response එකට levels දෙකකට වඩා ඇති විට e1071::svm() ස්වයංක්‍රීයව one-vs-one භාවිතා කරයි.

විය හැකි විභාග කාර්යය

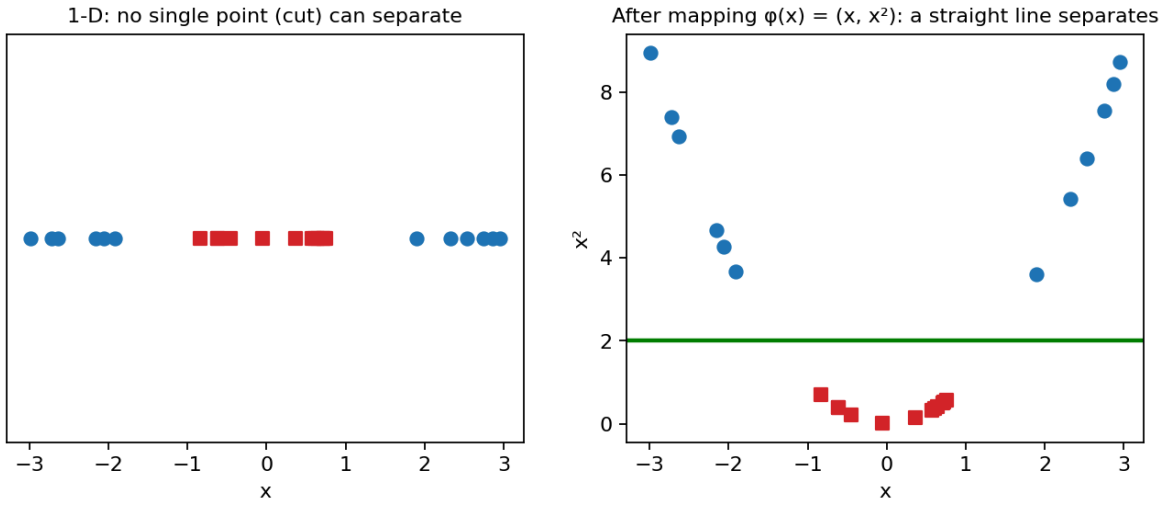
ඔබට dataset එකක් දී **වරකට classes දෙකක්** ගෙන model එක train කරන්න කියන්න පුළුවන්: classes දෙකක් වෙන් කර, එක් එක් SVM එක සාදා, ඉන්පසු majority vote එකකින් ඒකාබද්ධ කරන්න.

8. Non-linear SVM සහ kernel trick

සාමාන්‍ය SVM එකක් **linear** මායිමක් සොයයි. සැබෑ මායිම **වක්‍ර** නම්? එවිට සෘජු ඉරක් භයානක ලෙස නරක වැඩක් කරයි – කාසියක් උඩ දැමීමටත් වඩා නරක විය හැක (සමබර classes දෙකක් සඳහා random classifier එකක් 50% පමණ දෙයි). අපි හිර වුණාද? නැහැ.

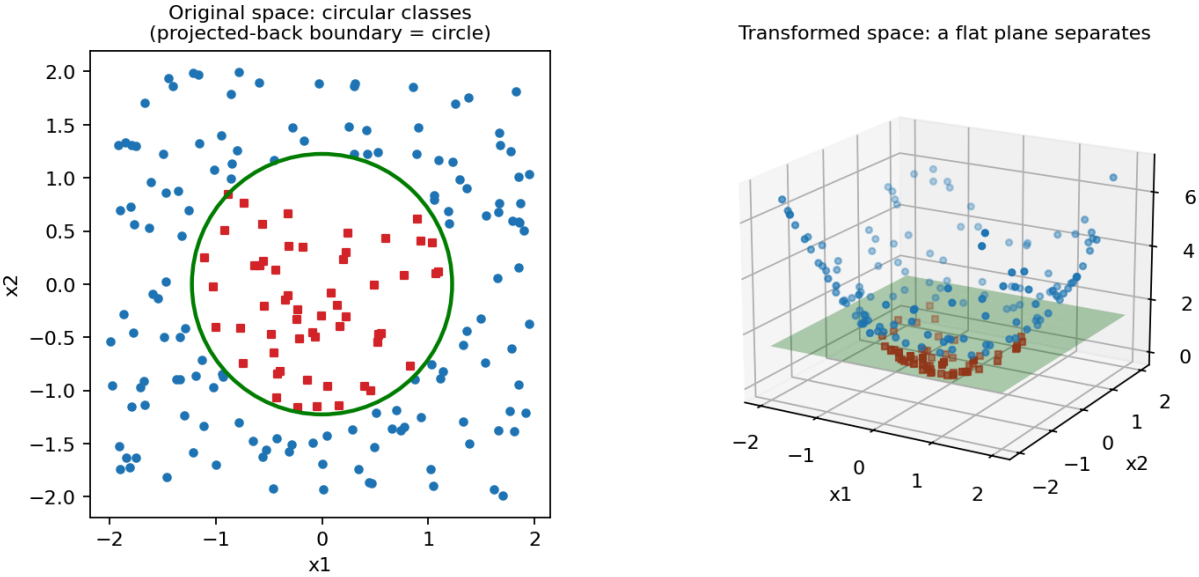
8.1 අදහස: දත්ත පරිවර්තනය කිරීම

අපි non-linear function $\varphi(x)$ එකක් මගින් දත්ත වඩා පොහොසත් අවකාශයකට **පූර්ව සැකසීම (pre-process)** / project කරනවා. එහි ඇති සෘජු වෙන් කරන්නා (hyperplane) මුල් අවකාශයේ **වක්‍ර** මායිමක් බවට පත් වේ.



1-D උදාහරණය: x^2 නව feature එකක් ලෙස එකතු කළ පසු, එක් සෘජු ඉරක් classes වෙන් කරයි

පන්තියේ පෙන්වූ වීඩියෝව ("SVM with a polynomial kernel"): තලයක ඇති රතු සහ නිල් points ඉරකින් වෙන් කළ නොහැක. Polynomial පරිවර්තනයකින් පසු points 3-D හි වක්‍ර (paraboloid) මතුපිටකට ඔසවනු ලැබේ; දැන් පැතලි **තලයක්** වර්ණ දෙක වෙන් කරයි. එම තලය නැවත පහළට project කළ විට මුල් තලයේ **රවුමක්** ලැබේ. අපි දත්ත වෙනස් කරන්නේ නැහැ – වෙනත් අවකාශයකින් බලනවා පමණයි.



$z = x1^2 + x2^2$ වෙත map කළ පසු රවුම් classes පැතලි තලයකින් වෙන් කළ හැක

8.2 Non-linear SVM – දැන් පියවර හතරයි

1. Non-linear (kernel) function එකකින් දත්ත **පූර්ව සැකසීම**.
2. Support vectors හඳුනා ගැනීම (පරිවර්තිත අවකාශයේ).
3. සමදුර ඉර/hyperplane ඇඳීම (පරිවර්තිත අවකාශයේ).
4. **මායිම නැවත මුල් අවකාශයට project කිරීම** – පරිවර්තිත අවකාශයේ ප්‍රතිඵල කිසිවෙකුට තේරුම් ගත නොහැකි නිසා මුල් variables වෙත නැවත එනවා. අපට **non-linear decision boundary** එකක් ලැබේ.

8.3 Kernel යනු කුමක්ද? (පන්ති ක්‍රියාකාරකම)

සන්දර්භය	තේරුම
Data science නොවන	යමක **හරය (core)**. Computing වලදී kernel එක යනු hardware සහ software ස්ථර අතර පාලමක් ලෙස ක්‍රියා කරන operating system එකේ මූලික කොටසයි. (තවද: ඇටයක/ධාන්‍යයක ඇතුළත කොටස.)
Data science / ML	**Kernel function** K(x, z) මගින් observations දෙකක් අතර සමානකම මනියි. එය පරිවර්තිත vectors සැබවින්ම සාදා **නොගෙන** ඒවායේ inner product ගණනය කරයි: $K(x, z) = \varphi(x)^T\varphi(z)$.

විශාල (අනන්ත වුවද) feature map එකක් කෙලින්ම සෑදීම ඉතා වියදම් අධිකයි. **Kernel trick** එකෙන් එම ප්‍රතිඵලයම ලාභදායීව ලැබේ. Non-linear decision function එක:

$$f(x) = \Sigma_{i \in S}\, \alpha_i\, y_i\, \mathbf{K(x_i, x)} + b$$

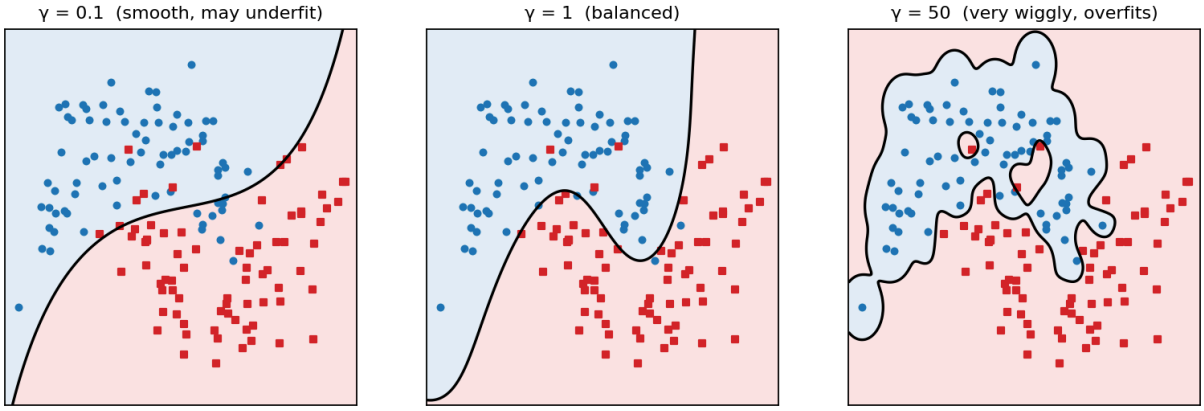
උදාහරණ map එකක්: $\varphi(x_1, x_2) = (x_1, x_2, x_1^2, x_2^2, x_1x_2)$ වක්‍ර මායිම් වලට ඉඩ දෙයි.

8.4 සුලබ kernels

Kernel	සූත්‍රය	R (e1071)	භාවිතා කරන්නේ කවදාද
Linear	$K(x, z) = x^T z$	kernel = "linear"	Linear මායිමක් පිළිගත හැකි විට; දැනටමත් predictors ගොඩක් ඇති විට (උදා: text)
Polynomial	$K(x, z) = (\gamma\, x^T z + r)^d$	kernel = "polynomial" (degree, gamma, coef0)	Interaction order එකට අර්ථයක් ඇති විට; ඉහළ d = වැඩි වක්‍රතාව
Radial basis (RBF / Gaussian)	$K(x, z) = \exp(-\gamma\|x - z\|^2)$	kernel = "radial" (default)	හොඳ සාමාන්‍ය non-linear තේරීමක්; ප්‍රාදේශීය සමානකම
Sigmoid	$K(x, z) = \tanh(\gamma\, x^T z + r)$	kernel = "sigmoid"	Neural-network activation එකක් වගේ; සැමවිටම වලංගු kernel එකක් නොවේ

RBF kernel හි γ (gamma) parameter

- **කුඩා γ**: සමානකම සෙමින් අඩු වේ → එක් එක් support vector එක විශාල ප්‍රදේශයකට බලපායි → **සුමට, අඩු නම්‍යශීලී** මායිම.
- **විශාල γ**: සමානකම ඉක්මනින් අඩු වේ → එක් එක් support vector එක කුඩා අසල්වැසි ප්‍රදේශයකට පමණක් බලපායි → **ප්‍රාදේශීය, රැළි සහිත** මායිම (overfit විය හැක).
- **C සහ γ එකට tune කළ යුතුයි**.



RBF kernel එකක් සමඟ γ හි බලපෑම (C ස්ථාවරයි)

8.5 වලංගු kernel එකක් යනු කුමක්ද?

$x_1, \ldots, x_n$ inputs සඳහා **Gram matrix** K හි ඇතුළත් කිරීම් $K_{ij} = K(x_i, x_j)$. සම්මත ප්‍රමාණවත් කොන්දේසිය: K **symmetric** සහ **positive semi-definite** විය යුතුයි, එනම් සෑම real vector c සඳහාම $c^T K c \geq 0$. එවිට kernel එක ඇත්තටම යම් feature space එකක inner product එකකි. (Exercise 4 බලන්න.)

8.6 Kernel එකක් තෝරා ගැනීම

- Baseline එකක් ලෙස **linear** kernel එකකින් පටන් ගන්න.
- **RBF** ප්‍රයෝජනවත් සාමාන්‍ය non-linear තේරීමකි (non-linear regression සඳහා ස්වභාවික තේරීම).
- Interaction order එකට අර්ථයක් ඇති විට **polynomial**.

- තෝරන්න **resampling (CV) කාර්ය සාධනය** අනුව, කිසිවිටක training accuracy පමණක් අනුව නොවේ.

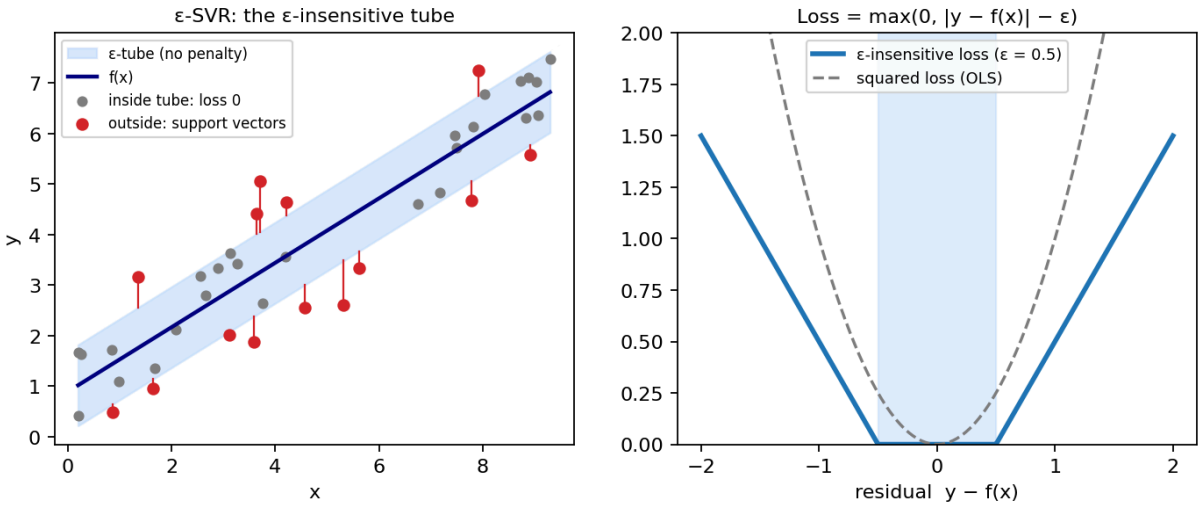
9. Support Vector Regression (ε-SVR)

Classification මගින් **කාණ්ඩ** පුරෝකථනය කරයි; regression මගින් **සංඛ්‍යා** පුරෝකථනය කරයි. SVR මූලික වශයෙන් එකම ක්‍රියාපටිපාටිය අනුගමනය කරයි:

1. **Support vectors හඳුනා ගැනීම** – නමුත් දැන් ඒවා **සම්පූර්ණ** දත්තවල "පිටත points" (වෙන් කිරීමට classes නැහැ).
2. **"මැද" ඉර ඇඳීම** – support vectors අතර දුර උපරිම කරන optimal hyperplane එක (regression function එක). (Kernel එකක් භාවිතා කරන්නේ නම් තවත් පියවර කිහිපයක්.)

9.1 ε-insensitive tube එක

ε-SVR සොයන්නේ **හැකි තරම් පැතලි** function එකක් $f(x) = w^Tx + b$, ε දක්වා errors **කිසිදු දඬුවමක් නැතිව** ඉඩ දෙමින්. $|y_i - f(x_i)| \le \varepsilon$ ප්‍රදේශය **ε-insensitive tube** එකයි.



වම: tube එක ඇතුළත points වලට වියදමක් නැහැ; පිටත points support vectors වේ. දකුණ: ε-insensitive loss සහ සාමාන්‍ය squared loss.

$$L_\varepsilon(y, f(x)) = \max\{0, |y - f(x)| - \varepsilon\}$$

Tube එක ඇතුළත errors වලට loss එක ශූන්‍යයි. එයින් පිටත points පමණක් loss එකට බලපාන අතර support vectors විය හැක.

9.2 Primal formulation

Slack variables දෙකක් භාවිතා කරනවා: ξ_i (tube එකට ඉහළින් point එක) සහ ξ_i^* (tube එකට පහළින්):

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \sum (\xi_i + \xi_i^*)$$

$$\text{subject to } y_i - (w^Tx_i + b) \le \varepsilon + \xi_i,\ (w^Tx_i + b) - y_i \le \varepsilon + \xi_i^*,\ \xi_i, \xi_i^* \ge 0$$

Kernel සහිත prediction: $f(x) = \sum (\alpha_i - \alpha_i^*) K(x_i, x) + b$. $\alpha_i - \alpha_i^* \neq 0$ ඇති points පමණක් දායක වේ.

9.3 SVR හි tuning parameters

Parameter	ප්‍රධාන බලපෑම
C	Tube එකෙන් ඔබ්බට errors සඳහා දඬුවම; විශාල C → training දත්තවලට හොඳින් fit වීමට දැඩි උත්සාහය
ε (epsilon)	දඬුවම් නැති tube එකේ පළල; විශාල ε → අඩු support vectors, සුමට fit එකක්
γ (gamma)	RBF kernel හි ප්‍රාදේශීයත්වය / නම්‍යශීලීත්වය

සැබෑ R පරීක්ෂාව (sine දත්ත, RBF, C = 10, γ = 1): ε වැඩි වන විට support vectors ගණන අඩු වේ:

ε	0.05	0.10	0.20	0.30	0.50
Support vectors ගණන (165 න්)	141	120	75	37	14

ඉතා කුඩා ε එකක් prediction ස්වයංක්‍රීයව වැඩි දියුණු **නොකරයි** – එය support vectors ගණන සහ noise වලට සංවේදීතාව වැඩි කරයි.

10. දත්ත සකස් කිරීම, tuning සහ ඇගයීම

10.1 Scaling

SVM **දුර සහ inner products** භාවිතා කරන නිසා, විශාල scale එකකින් මනින variable එකක් (උදා: රුපියල් වලින් ආදායම) කුඩා scale එකක variable එකක් (උදා: වයස) අභිබවා යයි. එබැවින් **standardise** කරන්න:

$$z_{ij} = (x_{ij} - \bar{x}_j) / s_j$$

Mean සහ sd ගණනය කරන්න **training දත්ත වලින් පමණක්**, ඉන්පසු එම පරිවර්තනයම validation/test දත්තවලට යොදන්න. e1071::svm() default ලෙස සංඛ්‍යාත්මක predictors scale කරයි (scale = TRUE).

10.2 Leakage නැතිව tuning

- C, γ, ε තෝරා ගැනීමට **training දත්ත ඇතුළත** cross-validation භාවිතා කරන්න.
- අවසාන model එක ස්ථිර වන තුරු **test set එක ස්පර්ශ නොකරන්න**.
- සම්පූර්ණ තේරීමේ ක්‍රියාවලිය සඳහා අපක්ෂපාතී ඇස්තමේන්තුවක් අවශ්‍ය නම් **nested cross-validation** භාවිතා කරන්න.
- ප්‍රයෝජනවත් අගයන් විශාලත්ව අනුපිළිවෙලින් (orders of magnitude) වෙනස් වන නිසා **logarithmic grid** එකක් භාවිතා කරන්න (උදා: $C = 2^{-1}, \ldots, 2^{5}$). රළු grid එකකින් පටන් ගෙන පසුව ශෝධනය කරන්න.

10.3 ඇගයීමේ මිනුම්

Classification	Regression
Confusion matrix; accuracy හෝ balanced accuracy; sensitivity සහ specificity; precision, recall, F1; ROC AUC (scores/probabilities ඇත්නම්)	$RMSE = \sqrt{(1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$ $MAE = (1/n)\ \Sigma\ \lvert y_i - \hat{y}_i \rvert$ $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$

අසමබර classes සඳහා accuracy නොමඟ යවන සුළු විය හැක (උදා: අවස්ථාවලින් 95% ක් "රෝගය නැත" – සැමවිටම "නැත" කීමෙන් 95% ලැබේ). වැරදිවල සැබෑ වියදම පිළිබිඹු කරන මිනුමක් තෝරන්න.

11. R demo: SVM classification

Package එක: **e1071** (svm() function එක). පන්තියේදී දේශකතුමා **caret** ද load කළා (දත්ත split කිරීමට ප්‍රයෝජනවත්, උදා: createDataPartition()). අවශ්‍ය නම් එක් වරක් install කරන්න.

11.1 Binary classification: setosa vs versicolor (linear kernel)

```
# install.packages("e1071")        # once
library(e1071)
set.seed(2026)
iris_bin <- subset(iris, Species != "virginica",
                   select = c(Petal.Length, Petal.Width, Species))
iris_bin$Species <- droplevels(iris_bin$Species)   # remove the empty level

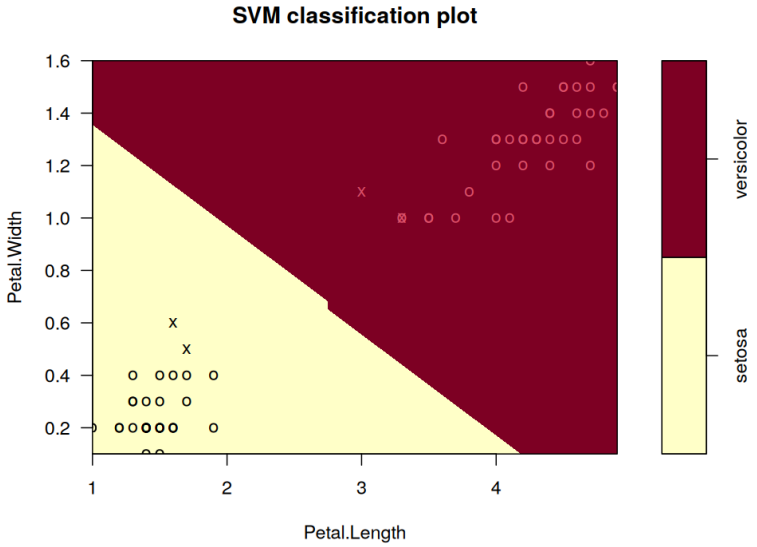
id <- sample(seq_len(nrow(iris_bin)), size = 0.70 * nrow(iris_bin))
train_cls <- iris_bin[id, ]
test_cls  <- iris_bin[-id, ]

fit_linear <- svm(Species ~ ., data = train_cls,
                  kernel = "linear", cost = 1, scale = TRUE)
fit_linear
pred_linear <- predict(fit_linear, newdata = test_cls)
table(Observed = test_cls$Species, Predicted = pred_linear)
mean(pred_linear == test_cls$Species)
```

```
Parameters:
   SVM-Type:  C-classification
 SVM-Kernel:  linear
       cost:  1
Number of Support Vectors:  4

            Predicted
Observed     setosa versicolor
  setosa         17          0
  versicolor      0         13
[1] 1                         # test accuracy = 100%
```

Training මල් 70 න් **support vectors 4 ක්** පමණක් මායිම තීරණය කරයි. Setosa සහ versicolor වෙන් කිරීම පහසු නිසා accuracy එක 100% යි.



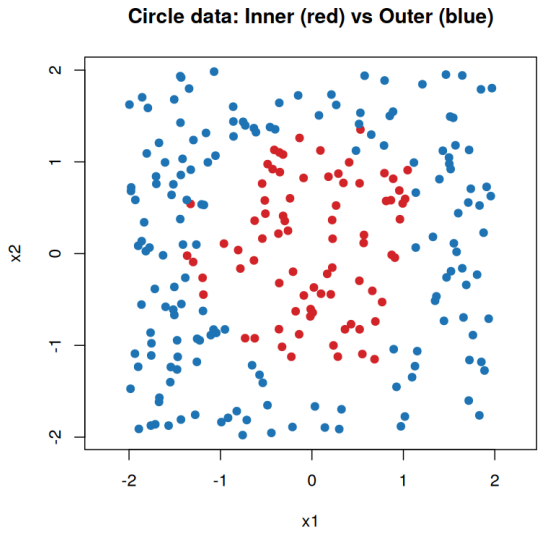
plot(fit_linear, train_cls, Petal.Width ~ Petal.Length). පසුබිම් වර්ණය = පුරෝකථනය කළ class එක; x = support vectors, o = අනෙක් points; සංකේතයේ වර්ණය = සැබෑ class එක.

11.2 Non-linear දත්ත: රවුමක් (linear vs RBF)

```
set.seed(2026)
n <- 240
x1 <- runif(n, -2, 2)
x2 <- runif(n, -2, 2)
radius2 <- x1^2 + x2^2
class_y <- factor(ifelse(radius2 + rnorm(n, 0, 0.30) > 1.7, "Outer", "Inner"))
circle_dat <- data.frame(x1, x2, class_y)
id <- sample(seq_len(n), size = 0.75 * n)
train_circle <- circle_dat[id, ];  test_circle <- circle_dat[-id, ]

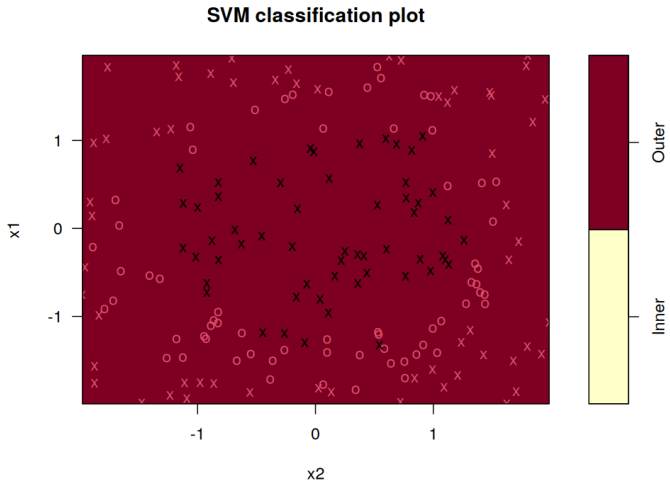
fit_circle_linear <- svm(class_y ~ ., data = train_circle, kernel = "linear", cost = 1)
fit_circle_rbf    <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                         cost = 10, gamma = 1)
pred_lin <- predict(fit_circle_linear, test_circle)
pred_rbf <- predict(fit_circle_rbf, test_circle)
c(linear_accuracy = mean(pred_lin == test_circle$class_y),
  rbf_accuracy    = mean(pred_rbf == test_circle$class_y))
plot(fit_circle_rbf, train_circle)
```

```
linear_accuracy    rbf_accuracy
      0.6166667       0.9666667
# support vectors: linear 115, RBF 33
```

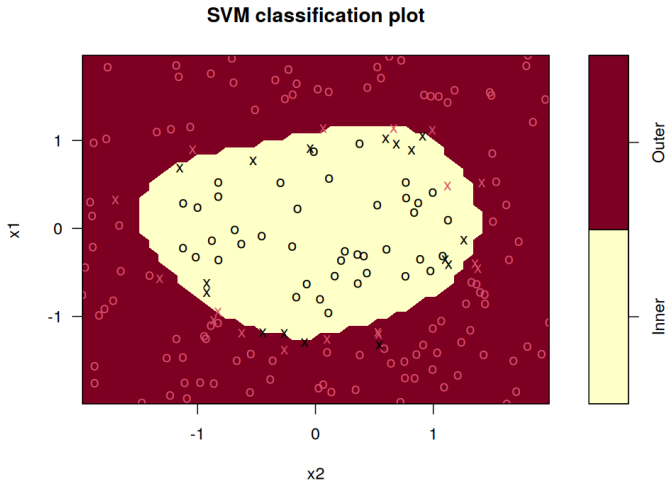


රවුම් දත්ත: Inner (රතු) ඇතුළත, Outer (නිල්) පිටත

Linear SVM ලබා ගන්නේ **61.7%** ක් පමණයි – සැමවිටම "Outer" යැයි අනුමාන කිරීමෙන් ලැබෙන අගයමයි (test points 60 න් 37 ක් Outer). එබැවින් සෘජු ඉර මෙහි නිෂ්ඵලයි. එයට හොඳ ඉරක් සොයා ගත නොහැකි නිසා support vectors 115 ක් අවශ්‍ය වේ. RBF kernel එක support vectors 33 කින් පමණක් **96.7%** ලබා ගනී.



Linear kernel: සෘජු මායිමකට රවුමක් ග්‍රහණය කළ නොහැක



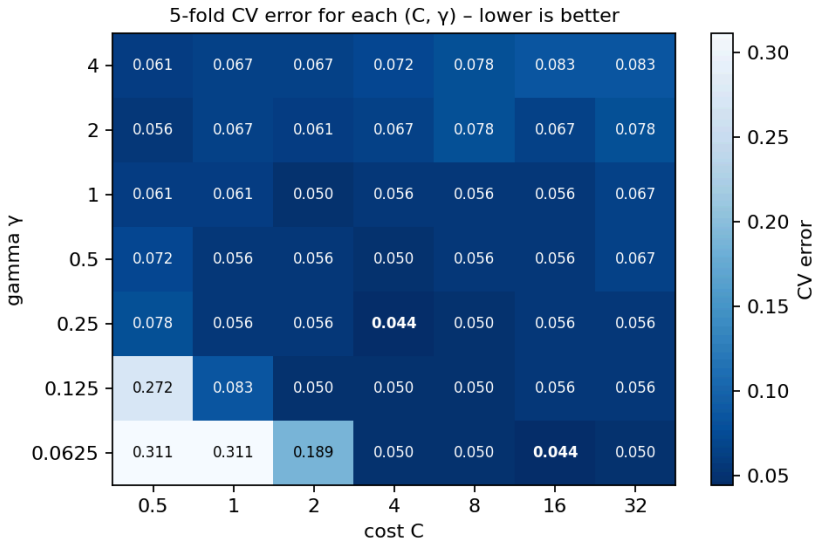
RBF kernel: මායිම ඇතුළත class එක වටා ඇති සංවෘත වක්‍රයකි

11.3 5-fold CV මගින් C සහ γ tune කිරීම

```
set.seed(2026)
tuned_cls <- tune(svm, class_y ~ ., data = train_circle, kernel = "radial",
                  ranges = list(cost = 2^(-1:5), gamma = 2^(-4:2)),
                  tunecontrol = tune.control(cross = 5))
summary(tuned_cls)
best_cls  <- tuned_cls$best.model
pred_best <- predict(best_cls, newdata = test_circle)
table(Observed = test_circle$class_y, Predicted = pred_best)
mean(pred_best == test_circle$class_y)
```

```
- best parameters:
 cost  gamma
   16 0.0625
- best performance: 0.04444444        # CV error = 4.4%

        Predicted
Observed Inner Outer
   Inner    21     2
   Outer     0    37
[1] 0.9666667
```



සංයෝජන 49 ටම CV error (costs 7 × gammas 7). Grid එක logarithmic.

tune() විසින් සංයෝජන 7 × 7 = 49 ක් උත්සාහ කළා, එක් එක් එක 5-fold CV සමඟ. හොඳම එකේ CV error 4.4% (සංයෝජන කිහිපයක්ම බොහෝ දුරට එතරම්ම හොඳයි). Tune කළ model එකේ test accuracy 96.7%.

11.4 Decision values සහ probabilities

SVM output එක probability එකක් **නොවේ**. decision.values = TRUE මගින් ලකුණ සහිත scores (දුර වැනි) ලැබේ. probability = TRUE මගින් probabilities ඇස්තමේන්තු කිරීමට e1071 අමතර **calibration** පියවරක් fit කරයි.

```
fit_prob <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                cost = 10, gamma = 1, probability = TRUE)
pred_prob <- predict(fit_prob, test_circle, probability = TRUE)
head(attr(pred_prob, "probabilities"))
```

```
         Outer       Inner
1 0.9954401733 0.004559827
2 0.0002168569 0.999783143
3 0.9961027888 0.003897211
...
# decision values (Outer/Inner): 3.11, -4.93, 3.20, ...  (positive -> Outer)
```

හොඳින් calibrate වූ අවදානම් ඇස්තමේන්තු වැදගත් නම් (උදා: වෛද්‍ය අවදානම), calibration එකම පරීක්ෂා කරන්න.

11.5 Class weights (අසමබර classes)

```
fit_weighted <- svm(class_y ~ ., data = train_circle, kernel = "radial",
                    cost = 10, gamma = 1,
                    class.weights = c(Inner = 2, Outer = 1))
```

දැන් "Inner" class එකේ වැරදි දෙගුණයක් මිලයි. අපේ run එකේ Inner හි test errors 2 සිට 1 දක්වා අඩු වුණා. Weights යාන්ත්‍රිකව තෝරා නොගෙන, වැරදිවල සැබෑ වියදම පිළිබිඹු කළ යුතුයි.

12. R demo: Support Vector Regression

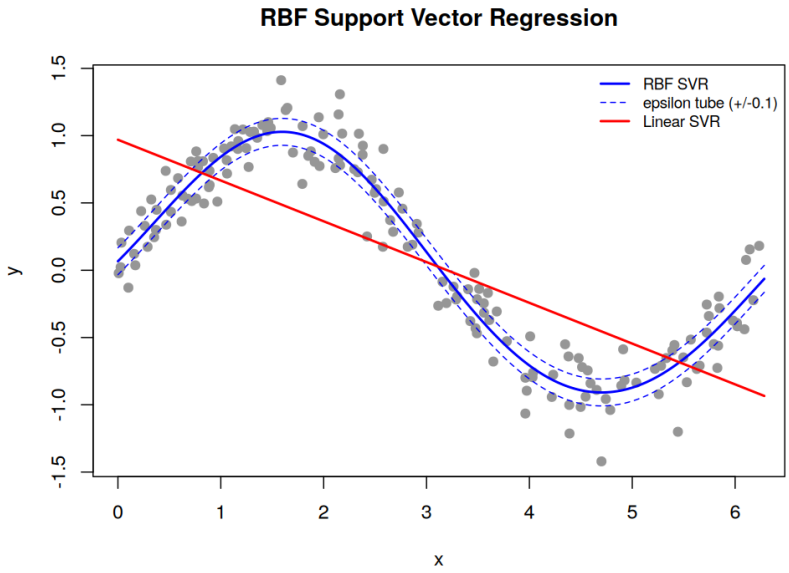
```
library(e1071)
set.seed(2026)
n <- 220
x <- runif(n, 0, 2 * pi)
y <- sin(x) + rnorm(n, mean = 0, sd = 0.18)     # non-linear truth: sine curve
reg_dat <- data.frame(x, y)
id <- sample(seq_len(n), size = 0.75 * n)
train_reg <- reg_dat[id, ];  test_reg <- reg_dat[-id, ]

svr_linear <- svm(y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "linear", cost = 10, epsilon = 0.10)
svr_rbf    <- svm(y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "radial", cost = 10, gamma = 1, epsilon = 0.10)
pred_linear <- predict(svr_linear, test_reg)
pred_rbf    <- predict(svr_rbf, test_reg)

rmse <- function(actual, predicted) sqrt(mean((actual - predicted)^2))
mae  <- function(actual, predicted) mean(abs(actual - predicted))
rbind(Linear_SVR = c(RMSE = rmse(test_reg$y, pred_linear), MAE = mae(test_reg$y, pred_linear)),
      RBF_SVR    = c(RMSE = rmse(test_reg$y, pred_rbf),    MAE = mae(test_reg$y, pred_rbf)))

x_grid <- data.frame(x = seq(0, 2 * pi, length.out = 300))
y_grid <- predict(svr_rbf, newdata = x_grid)
plot(train_reg$x, train_reg$y, pch = 19, col = "grey60",
     xlab = "x", ylab = "y", main = "RBF Support Vector Regression")
lines(x_grid$x, y_grid, col = "blue", lwd = 2)
```

```
                RMSE       MAE
Linear_SVR 0.4139238 0.3566986
RBF_SVR    0.1421773 0.1109824
```



RBF SVR sine වක්‍රය අනුගමනය කරයි; linear SVR ට ඇඳිය හැක්කේ සෘජු ඉරක් පමණයි. (Dashed නිල් = ε-tube, පැහැදිලි කිරීමට එකතු කළා.)

සැබෑ සම්බන්ධතාවය non-linear නිසා RBF SVR error එක linear SVR error එකෙන් **තුනෙන් එකක්** පමණයි. සටහන: noise sd එක 0.18 යි, එබැවින් 0.14 ක RMSE එකක් ලබා ගත හැකි හොඳම අගයට ආසන්නයි.

12.1 C, γ සහ ε tune කිරීම

```
set.seed(2026)
tuned_reg <- tune(svm, y ~ x, data = train_reg, type = "eps-regression",
                  kernel = "radial",
                  ranges = list(cost = c(0.1, 1, 10, 100),
                                gamma = c(0.05, 0.1, 0.5, 1, 2),
                                epsilon = c(0.05, 0.10, 0.20, 0.30)),
                  tunecontrol = tune.control(cross = 5))
summary(tuned_reg)
best_reg   <- tuned_reg$best.model
pred_tuned <- predict(best_reg, newdata = test_reg)
c(RMSE = rmse(test_reg$y, pred_tuned), MAE = mae(test_reg$y, pred_tuned))
```

```
- best parameters:  cost = 10, gamma = 1, epsilon = 0.3
- best performance (CV MSE): 0.0328
     RMSE       MAE
0.1508785 0.1186085        # only 37 support vectors
```

Tune කළ model එක **පළල් tube** එකක් (ε = 0.3) තෝරා ගත්තා → support vectors 37 ක් පමණයි, test error එක බොහෝ දුරට එකමයි (මෙම test set එකේ ටිකක් වැඩියි). බොහෝ දුරට එතරම්ම නිවැරදි සරල model එකක් සාමාන්‍යයෙන් වඩා කැමතියි. දැඩි සැසඳීමක් සඳහා, සියලු models **එකම** resampling splits මත ඇගයීමට ලක් කරන්න.

13. පන්තියේ ක්‍රියාකාරකම්

13.1 ක්‍රියාකාරකම A – iris මත SVM: versicolor vs virginica

කාර්යය: (1) iris load කරන්න; **versicolor** සහ **virginica** පමණක් තබා ගන්න (setosa ඉවත් කරන්න); (2) default arguments භාවිතයෙන් Species ~ Petal.Length + Petal.Width SVM එකක් fit කරන්න (kernel = "linear" ද උත්සාහ කළ හැක); (3) දත්ත split කර (80/20) test කරන්න.

```
library(caret)     # useful for splitting (createDataPartition)
library(e1071)
data(iris)
dim(iris)                                              # 150 5

data_filtered <- iris[iris$Species != "setosa", ]   # or subset(iris, Species != "setosa")
data_filtered$Species <- factor(data_filtered$Species)  # drop the unused level
dim(data_filtered)                                     # 100 5
table(data_filtered$Species)

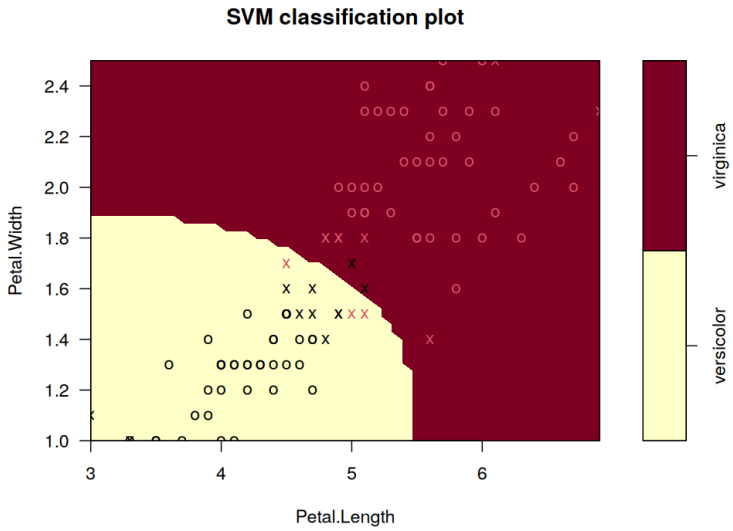
m0 <- svm(Species ~ Petal.Length + Petal.Width, data = data_filtered)  # defaults
m0
```

```
[1] 150   5
[1] 100   5
versicolor  virginica
        50         50
   SVM-Type:  C-classification
 SVM-Kernel:  radial            # default kernel
       cost:  1                 # default cost
Number of Support Vectors:  23
```

දේශකතුමා ප්‍රශංසා කළ දේවල් දෙකක්

• Filter කළ පසු Species factor() මගින් පරිවර්තනය කිරීම – නැත්නම් හිස් "setosa" level එක ඉතිරි වන අතර svm() එය class 3 ගැටලුවක් ලෙස සැලකිය හැක.

• Model කිරීමට පෙර දත්ත split කිරීම (train/test). Dimension පරීක්ෂාව (150 → 100) මගින් setosa ඉවත් වූ බව තහවුරු වේ.



Versicolor vs virginica සඳහා default (radial) SVM – classes overlap වන නිසා setosa ට වඩා අමාරුයි

Homework: මුලින්ම split කර, SVM, logistic regression සහ random forest සසඳන්න

```
library(randomForest)
set.seed(123)
ids   <- sample(1:nrow(data_filtered), 0.8 * nrow(data_filtered), replace = FALSE)
train <- data_filtered[ids, ];  test <- data_filtered[-ids, ]

m_svm <- svm(Species ~ Petal.Length + Petal.Width, data = train, kernel = "linear")
m_log <- glm(Species ~ Petal.Length + Petal.Width, data = train, family = binomial)
m_rf  <- randomForest(Species ~ Petal.Length + Petal.Width, data = train)

p_svm <- predict(m_svm, test)
p_log <- ifelse(predict(m_log, test, type = "response") > 0.5, "virginica", "versicolor")
p_rf  <- predict(m_rf, test)
c(SVM = mean(p_svm == test$Species), Logistic = mean(p_log == test$Species),
  RandomForest = mean(p_rf == test$Species))
```

```
        SVM    Logistic RandomForest
       0.95        0.95         0.95      # 19 of 20 test flowers correct
```

මෙම කුඩා test set එකේ (මල් 20) තුනම 95% දෙයි – එක් එක් එකේ එක වැරැද්දක්. Test අවස්ථා මෙතරම් අඩු නිසා, ඒවා වඩා විශ්වාසදායක ලෙස සැසඳීමට cross-validation භාවිතා කරන්න. (වෙනත් seed එකක් සමඟ ඔබේ අගයන් වෙනස් විය හැක.)

13.2 ක්‍රියාකාරකම B – mtcars: linear regression vs SVR

කාර්යය: mtcars භාවිතා කරන්න; **wt** සහ **hp** භාවිතයෙන් **mpg** පුරෝකථනය කරන්න; (1) linear regression model එකක් සාදන්න; (2) SVR model එකක් සාදන්න; (3) test set එකේ RMSE

සසඳන්න (80/20 split).

svm() help page එකේ සටහන්

- svm() මගින් සාමාන්‍ය **regression සහ classification** (සහ density estimation) කළ හැක.
- Default **type** එක C-classification. **Type එක වෙනස් නොකළොත් එය classification කරයි!** Regression සඳහා `type = "eps-regression"` අවශ්‍යයි. (ඇත්තටම response එක සංඛ්‍යාත්මක නම් e1071 ස්වයංක්‍රීයව eps-regression වෙත මාරු වේ – නමුත් සැමවිටම පැහැදිලිව ලියන්න.)
- ලබා ගත හැකි kernels: `"linear", "polynomial", "radial", "sigmoid"`. Non-linear සම්බන්ධතා සඳහා **radial** kernel එක ස්වභාවික තේරීමයි.

```
set.seed(123)
ids <- sample(1:nrow(mtcars), 0.8 * nrow(mtcars), replace = FALSE)  # unique rows
train_data <- mtcars[ids, ]
test_data  <- mtcars[-ids, ]

lm_mtcars  <- lm(mpg ~ wt + hp, data = train_data)
svr_mtcars <- svm(mpg ~ wt + hp, data = train_data,
                  type = "eps-regression", kernel = "radial")

pred_lm  <- predict(lm_mtcars,  newdata = test_data)
pred_svr <- predict(svr_mtcars, newdata = test_data)

rmse <- function(pred, obs) {
  sq_err <- (pred - obs)^2
  sqrt(sum(sq_err) / length(obs))      # <- the LAST line is what R returns
}
rmse(pred_lm,  test_data$mpg)
rmse(pred_svr, test_data$mpg)
```

```
                   actual    lm   svr
Mazda RX4 Wag         21.0 22.80 21.55
Valiant               18.1 20.59 19.89
Merc 450SE            16.4 15.57 13.72
Merc 450SL            17.3 16.96 14.73
Lincoln Continental   10.4  8.88 14.00
Toyota Corona         21.5 24.91 24.45
Pontiac Firebird      19.2 16.66 14.49
 RMSE_lm RMSE_svr
2.090903 2.955185
```

sample() සමඟ `replace = FALSE` ඇයි? අපට **අනන්‍ය (unique)** row IDs අවශ්‍යයි; test set එක සරලව `-ids` (තෝරා නොගත් සියලු rows). සටහන: 0.8 × 32 = 25.6 → training කාර් 25 ක් සහ **test කාර් 7 ක්**.

ප්‍රතිඵලය: මෙහි linear model එකට **අඩු** RMSE එකක් ඇත (2.09 vs 2.96). පන්තියේ මෙන්ම, මෙම අවස්ථාවේ SVR වඩා නරකයි – සමහරවිට predictors දෙකක් පමණක් භාවිතා කළ නිසා, default C/γ/ε tune නොකළ නිසා, සහ sample එක ඉතා කුඩා නිසා. (Predictors 10 ම භාවිතා කළ විට: SVR RMSE 2.79 vs lm 2.59 – තවමත් වඩා හොඳ නැහැ.) වඩා නම්‍යශීලී model එකක් **ස්වයංක්‍රීයව** වඩා හොඳ නැහැ.

පන්තියේ ඇති වූ bug එක (R functions ගැන පාඩමක්)

දේශකතුමා මුලින්ම RMSE function එක ලියූ විට වැරදි අගයක් return කළා, මොකද R function එකක් return කරන්නේ එහි **අවසාන ඇගයූ ප්‍රකාශනයේ** අගයයි. අවසාන පේළිය assignment එකක් (උදා: `err <- ...`) හෝ අතරමැදි පියවරක් නම්, RMSE එක ලැබෙන්නේ නැහැ. විසඳුම: අවසාන පේළිය (හෝ පැහැදිලි `return()` එකක්) අවසාන RMSE එක බවට වග බලා ගන්න. දන්නා අගයකින් function එක එක් වරක් පරීක්ෂා කරන්න (උදා: rmse(c(1,2), c(1,4)) = √2 ≈ 1.414 විය යුතුයි).

14. සුලබ වැරදි සහ වෙනත් ක්‍රම සමඟ සැසඳීම

14.1 වළක්වා ගත යුතු ප්‍රායෝගික වැරදි

වැරැද්ද	එය ගැටලුවක් වන්නේ ඇයි
Predictors scale නොකිරීම	විශාල scale ඇති variables දුර සහ inner products අභිබවා යයි
Test set එක මත tune කිරීම	තොරතුරු කාන්දු වේ; අවසාන කාර්ය සාධනය ඕනෑවට වඩා ශුභවාදී වේ
Training accuracy අනුව parameters තෝරා ගැනීම	නම්‍යශීලී kernels (විශාල C, විශාල γ) overfit වේ
Non-linear kernel එකක් සැමවිටම වඩා හොඳ යැයි සිතීම	Linear SVM එකක් සරල, වේගවත් සහ ස්ථාවර විය හැක
Decision score එක probability එකක් ලෙස සැලකීම	Calibration අවශ්‍යයි (probability = TRUE)
Class අසමබරතාව නොසලකා හැරීම	Accuracy මගින් සුළුතර class එකේ දුර්වල කාර්ය සාධනය සැඟවේ – class weights සහ වඩා හොඳ මිනුම් භාවිතා කරන්න
විශාල grid එකක් සෙවීම	රළු log grid එකකින් පටන් ගෙන, හොඳ ප්‍රදේශය සොයා, පසුව ශෝධනය කරන්න
SVM ස්වයංක්‍රීයව outliers වලට ශක්තිමත් යැයි සිතීම	විශාල C එකක් මායිම අසාමාන්‍ය points වලට සංවේදී කරයි

14.2 SVM සහ වෙනත් ක්‍රම

අංශය	SVM	Logistic / linear regression	Tree-based ක්‍රම
මායිම	Linear හෝ kernel-based non-linear	Features සාදා නොමැති නම් සාමාන්‍යයෙන් linear	කොටස් වශයෙන් සෘජුකෝණාස්‍රාකාර, non-linear
Scaling අවශ්‍යද	සාමාන්‍යයෙන් ඔව්	බොහෝ විට සුදුසුයි	සාමාන්‍යයෙන් නැහැ
ස්වදේශීය probability	නැහැ	ඔව් (logistic)	බොහෝ විට ඇස්තමේන්තුගත probabilities දෙයි
අර්ථකථනය	මධ්‍යස්ථ (linear); අඩු (kernels)	සාපේක්ෂව ඉහළ	වෙනස් වේ
High-dimensional දත්ත	බොහෝ විට ශක්තිමත්	Regularisation සමඟ ශක්තිමත් විය හැක	වෙනස් වේ
ප්‍රධාන tuning	C, kernel parameters (γ, degree), SVR සඳහා ε	Penalty සහ mixing parameter (regularised නම්)	Depth, node size, trees ගණන

සෑම ගැටලුවකම ජය ගන්නා ක්‍රමයක් නැහැ. Predictive කාර්ය සාධනය, අර්ථකථනය, ගණනය කිරීම, calibration සහ වැරදිවල ප්‍රතිවිපාක අනුව තෝරන්න.

SVM විශේෂයෙන් ප්‍රයෝජනවත් වන්නේ කවදාද? Predictors බොහොමයක් ඇති විට; non-linear සම්බන්ධතා ඇති විට; පරිවර්තිත variables ගොඩක් ඔබම නොසාදා නම්‍යශීලී මායිමක් අවශ්‍ය විට; coefficients අර්ථකථනයට වඩා prediction වැදගත් විට.

15. Exercises – විසඳුම් සමඟ

Exercise 1 – දුර සහ class පුරෝකථනය

Decision function $f(x) = 2x_1 - x_2 - 3$. (a) w සහ b සඳහන් කරන්න. (b) $x_A = (3, 1)$ සහ $x_B = (1, 2)$ classify කරන්න. (c) ලකුණ සහිත ලම්බක දුර සොයන්න.

විසඳුම. $f(x) = w^Tx + b$ සමඟ සසඳන්න: $\mathbf{w = (2, -1)^T}$, $\mathbf{b = -3}$, සහ $\|w\| = \sqrt{(2^2 + (-1)^2)} = \sqrt{5} \approx 2.236$.

Point	f(x)	Class (ලකුණ)	ලකුණ සහිත දුර = f(x)/‖w‖
$x_A = (3, 1)$	$2(3) - 1 - 3 = 2$	+1	$2/\sqrt{5} \approx 0.894$
$x_B = (1, 2)$	$2(1) - 2 - 3 = -3$	−1	$-3/\sqrt{5} \approx -1.342$

Exercise 2 – Slack variables

$y_i f(x_i)$ අගයන් 1.4, 0.6, 0 සහ −0.3. අවම slack එක සොයා අර්ථකථනය කරන්න.

විසඳුම. $\xi_i = \max\{0, 1 - y_i f(x_i)\}$.

$y_i f(x_i)$	ξ_i	අර්ථකථනය
1.4	max(0, −0.4) = 0	නිවැරදි පැත්තේ, margin ඉරෙන් ඔබ්බට
0.6	0.4	නිවැරදිව classify වී ඇත, නමුත් margin එක ඇතුළත
0	1	Decision boundary එක මත
−0.3	1.3	වැරදියට classify වී ඇත

Exercise 3 – C සහ γ හි බලපෑම (RBF SVM)

- **C වැඩි කිරීම (γ ස්ථාවරව):** උල්ලංඝන වලට වැඩි දඬුවම් → පටු margin, training අවස්ථා නිවැරදිව classify කිරීමට වැඩි උත්සාහය.
- **γ වැඩි කිරීම (C ස්ථාවරව):** එක් එක් support vector එක කුඩා අසල්වැසි ප්‍රදේශයකට බලපායි → වඩා අක්‍රමවත්, ප්‍රාදේශීය මායිමක්.
- **C සහ γ දෙකම ඉතා විශාල:** noise/outliers අනුගමනය කරන ඉතා සංකීර්ණ මායිමක් → අඩු training error නමුත් දුර්වල generalisation (**overfitting**).
- මේවා සාමාන්‍ය ප්‍රවණතා මිස සහතික නොවේ – හැසිරීම දත්ත මත රඳා පවතී.

Exercise 4 – Kernel එකක් පරීක්ෂා කිරීම

$x_1 = 1$, $x_2 = 2$, $K(x, z) = xz + 1$. (a) Gram matrix. (b) එය positive semi-definite බව පෙන්වන්න. (c) Feature map එකක් දෙන්න.

(a) $K(1,1) = 1\cdot1 + 1 = 2$; $K(1,2) = K(2,1) = 1\cdot2 + 1 = 3$; $K(2,2) = 2\cdot2 + 1 = 5$. එබැවින් **K = [[2, 3], [3, 5]]**.

(b) Leading principal minors: $2 > 0$ සහ $\det(K) = 2\cdot5 - 3\cdot3 = 1 > 0$ → positive definite, එබැවින් positive semi-definite. Eigenvalues $\lambda = (7 \pm \sqrt{45})/2$ = **6.854 සහ 0.146** (දෙකම ධන – R හි eigen() මගින් පරීක්ෂා කළා).

(c) $\varphi(x) = (x, 1)^T$, මොකද $\varphi(x)^T\varphi(z) = x\cdot z + 1\cdot1 = xz + 1$.

Exercise 5 – ε-insensitive loss

$\varepsilon = 0.2$; residuals 0.10, −0.35, 0.80. Losses, tube එකෙන් පිටත points සහ මුළු එකතුව සොයන්න.

Residual r	|r|	Loss = max(0, |r| − 0.2)	Tube එකෙන් පිටතද?
0.10	0.10	0	නැහැ
−0.35	0.35	0.15	ඔව්
0.80	0.80	0.60	ඔව්

මුළු loss = 0 + 0.15 + 0.60 = **0.75**.

Exercise 6 – R classification කාර්යය (iris species තුනම)

```
library(e1071)
set.seed(2026)
id <- sample(seq_len(nrow(iris)), size = floor(0.70 * nrow(iris)))
train <- iris[id, ];  test <- iris[-id, ]

linear_fit  <- svm(Species ~ ., data = train, kernel = "linear", cost = 1, scale = TRUE)
radial_tune <- tune(svm, Species ~ ., data = train, kernel = "radial",
                    ranges = list(cost = 2^(-1:5), gamma = 2^(-5:1)),
                    tunecontrol = tune.control(cross = 5))
radial_fit  <- radial_tune$best.model

pred_linear <- predict(linear_fit, test);  pred_radial <- predict(radial_fit, test)
table(Observed = test$Species, Predicted = pred_linear)
table(Observed = test$Species, Predicted = pred_radial)
c(linear_accuracy = mean(pred_linear == test$Species),
  radial_accuracy = mean(pred_radial == test$Species))
c(linear_support_vectors = linear_fit$tot.nSV, radial_support_vectors = radial_fit$tot.nSV)
radial_tune$best.parameters
```

```
            Predicted                         (same table for linear and radial)
Observed     setosa versicolor virginica
  setosa         18          0         0
  versicolor      0         18         2
  virginica       0          0         7
linear_accuracy radial_accuracy
      0.9555556       0.9555556
linear_support_vectors radial_support_vectors
                    24                      31
  cost   gamma
     8 0.03125              # best CV error 2.9%
```

තේරීම: models දෙකම 95.6% (45 න් 43) ලබා ගන්නා අතර එකම වැරදි දෙක කරයි (versicolor, virginica ලෙස පුරෝකථනය කිරීම). Test set එක කුඩා නිසා මල් එකක් හෝ දෙකක් accuracy එක බොහෝ වෙනස් කරයි. දෙකම එකම ලෙස ක්‍රියා කරන නිසා **linear model** එක සාධාරණ තේරීමකි: එය සරලයි සහ අඩු support vectors භාවිතා කරයි (24 vs 31). පසුබිමේ: species 3 → one-vs-one sub-models 3.

Exercise 7 – R regression කාර්යය (mtcars, predictors 4)

```
set.seed(2026)
dat <- mtcars[, c("mpg", "wt", "hp", "disp", "qsec")]
id  <- sample(seq_len(nrow(dat)), size = floor(0.75 * nrow(dat)))
train <- dat[id, ];  test <- dat[-id, ]

linear_fit <- svm(mpg ~ ., data = train, type = "eps-regression",
                  kernel = "linear", cost = 1, epsilon = 0.1, scale = TRUE)
radial_tune <- tune(svm, mpg ~ ., data = train, type = "eps-regression", kernel = "radial",
                    ranges = list(cost = c(0.1, 1, 10, 100), gamma = c(0.01, 0.1, 0.5, 1),
                                  epsilon = c(0.05, 0.1, 0.2, 0.5)),
                    tunecontrol = tune.control(cross = 5))
radial_fit <- radial_tune$best.model
pred_linear <- predict(linear_fit, test);  pred_radial <- predict(radial_fit, test)
rmse <- function(y, p) sqrt(mean((y - p)^2));  mae <- function(y, p) mean(abs(y - p))
rbind(Linear = c(RMSE = rmse(test$mpg, pred_linear), MAE = mae(test$mpg, pred_linear)),
      RBF    = c(RMSE = rmse(test$mpg, pred_radial), MAE = mae(test$mpg, pred_radial)))
radial_tune$best.parameters
```

```
           RMSE      MAE
Linear 3.924017 3.012459
RBF    4.137543 3.583697
   cost gamma epsilon
     10  0.01     0.5
```

සාකච්ඡාව: කාර් 32 ක් පමණක් ඇති නිසා test set එකේ ඇත්තේ කාර් **8 ක්** පමණයි. සැසඳීම අස්ථාවරයි – වෙනත් split එකකින් අනුපිළිවෙල ප්‍රතිවිරුද්ධ විය හැක. මෙහි linear SVR ටිකක් හොඳයි. Repeated හෝ nested cross-validation වඩා විශ්වාසදායකයි; වැඩි දත්ත ඇත්නම් ස්වාධීන test set එකක් වඩා හොඳයි.

16. Neural Networks හැඳින්වීම

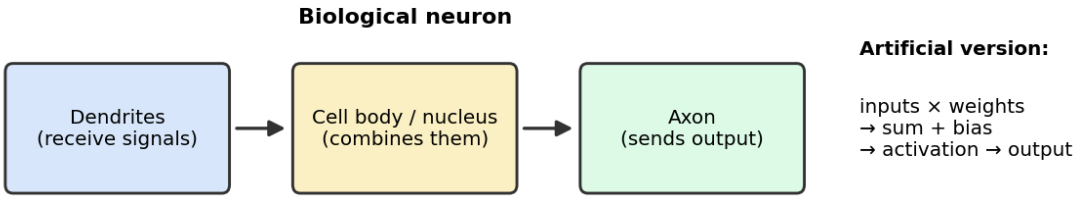
Syllabus එකේ පිහිටීම: unsupervised learning (clustering, dimension reduction, recommender systems) → supervised learning (parametric සහ non-parametric algorithms, **SVM සහ kernels**) → ඉතිරි වන්නේ: **neural networks**, ඉන්පසු deep learning. මෙතැන් සිට coding ගොඩක් තියෙනවා.

16.1 පන්ති ක්‍රියාකාරකම: neural network එකක ප්‍රධාන කොටස්

පන්තියේ ලැබුණු පිළිතුර: **input layer, hidden layer(s) සහ output layer**. අපි එකතු කරනවා: **weights**, **biases**, **activation functions**, **loss function** එකක් සහ **optimiser** එකක් (training ක්‍රමය).

යොමු පොත: **C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook***. පරිච්ඡේද: neural networks හැඳින්වීම; shallow neural networks සමඟ machine learning; deep neural networks train කිරීම; radial basis function networks; restricted Boltzmann machines; recurrent neural networks; convolutional neural networks; deep reinforcement learning; deep learning හි උසස් මාතෘකා.

16.2 ජීව විද්‍යාත්මක neuron සිට කෘත්‍රිම neuron දක්වා

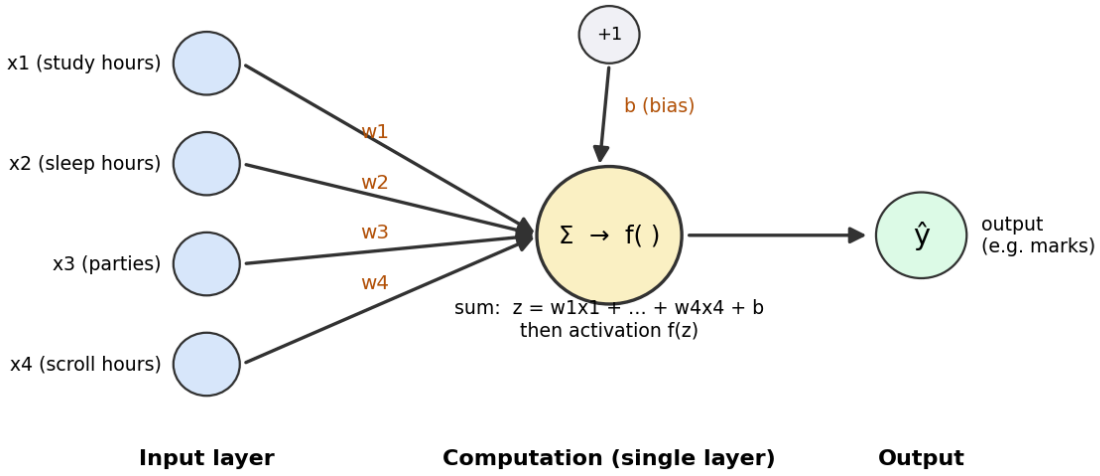


අපේ මොළයේ **neuron** එකක් තම **dendrites** හරහා සංඥා ලබා ගෙන, සෛල දේහයේ (nucleus) ඒවා ඒකාබද්ධ කර, **axon** හරහා output එකක් යවයි. විද්‍යාඥයින් මෙම අදහස ගණිතමය model එකකට පිටපත් කළා: **artificial neural network (ANN)**, **neurons** ලෙසද හඳුන්වන ගණනය කිරීමේ ඒකක වලින් සෑදී ඇත. ඒවා ජීව විද්‍යාත්මක නොවේ – කෘත්‍රිමව සාදන ලද ඒවා – නමුත් සාමාන්‍යයෙන් අපි "neural network" හෝ "neural net" කියනවා.

16.3 Perceptron (single-layer network)

Single-layer network එකක, inputs කට්ටලයක් linear function එකක සාමාන්‍යකරණය කළ අනුවාදයක් භාවිතයෙන් කෙලින්ම output එකකට map කරයි. මෙම සරලම neural network එක

perceptron ලෙස හැඳින්වේ – ANN එකක මූලික ඒකකය. (Perceptron එකක් අඳින්න කිව්වොත් මෙය අඳින්න පුළුවන් විය යුතුයි.)



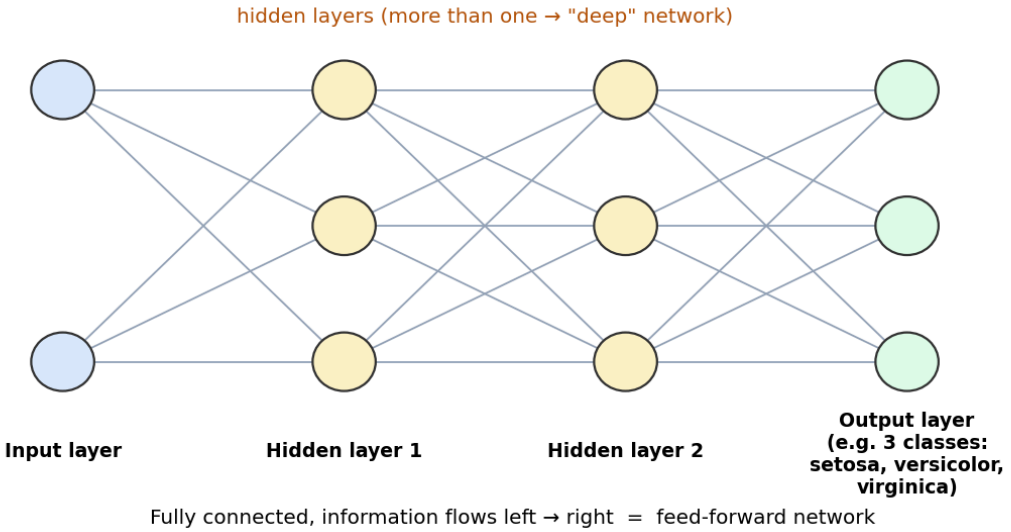
පන්ති උදාහරණය – group-project ලකුණු: ලකුණු රඳා පවතින්නේ පාඩම් කළ පැය (x_1), නිදාගත් පැය (x_2), සාද ගණන (x_3) සහ social media scroll කළ පැය (x_4) මත යැයි සිතමු. සමහර inputs උදව් කරයි (ධන weight), අනෙක් ඒවා හානි කරයි (සෘණ weight), උදා:

$$\text{marks} = 20.5\cdot(\text{study}) - 5\cdot(\text{sleep}) - 2.5\cdot(\text{parties}) - 1.5\cdot(\text{scrolling}) + b$$

මෙය හරියටම **multiple linear regression** එකකි: $y = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$. මැද node එක **එකතු කිරීම (summation)** කරයි. **Bias** b, regression $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \varepsilon$ හි β_0 ගේ භූමිකාව ඉටු කරයි (error පදය model එකෙන් අපගමනයන් හසුරුවයි). Bias එකට **input එකක් නැහැ** (එහි input එක නියත 1 ලෙස සිතන්න) නමුත් එයට **weight** එකක් ඇත.

16.4 Multi-layer networks

Multi-layer network එකක neurons ස්ථර (layers) වශයෙන් සකස් කර ඇත: input සහ output layers **hidden layers** එකක් හෝ කිහිපයක් මගින් වෙන් කර ඇත. මෙම ස්ථර අනුව සැකසූ ව්‍යුහය **feed-forward network** ලෙස හැඳින්වේ (තොරතුරු ඉදිරියට ගලා යයි, input → output).



- සාමාන්‍යයෙන් සෑම node එකක්ම ඊළඟ layer එකේ **සෑම** node එකකටම සම්බන්ධයි (fully connected). සම්බන්ධතාවයක් නැත්නම්, එම nodes අතර සම්බන්ධයක් නැහැ.
- Hidden layers එකකට වඩා → **deep neural network**.
- **Architecture** = hidden layers කීයද සහ එක් එක් එකේ nodes කීයද (උදා: input 1 – hidden nodes 2 – output 1; හෝ input – hidden – hidden – output). ගැඹුරු networks බොහෝ විට වඩා හොඳින් generalise වේ ("better generalisation with depth").

16.5 Neural network එකක ව්‍යුහය (anatomy)

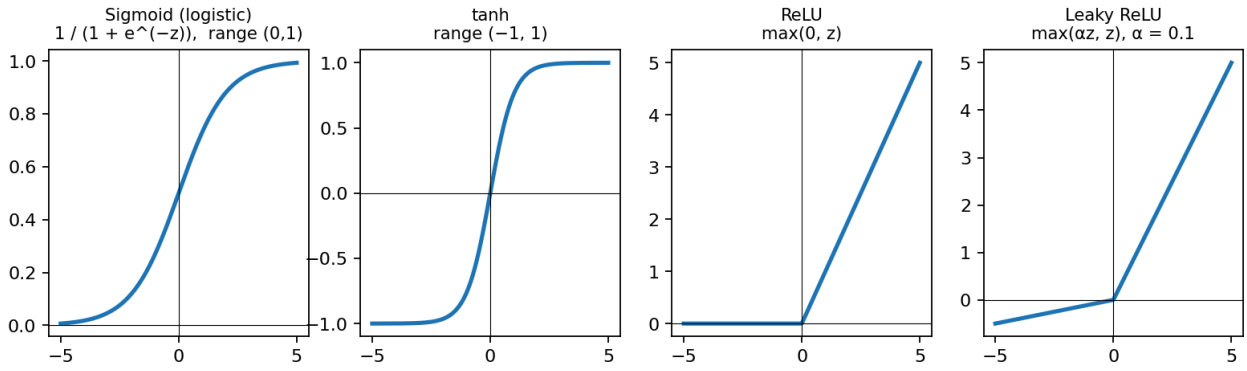
කොටස	භූමිකාව
Inputs සහ weights	සෑම input එකක්ම weight එකකින් ගුණ කරයි; bias එකක් එකතු කරයි
Activation function	එක් එක් node එක ඇතුළත function එක (බර සහිත එකතුවට යොදයි). Non-linearity ලබා දෙයි
Loss function	Predictions කොතරම් වැරදිද මනියි; training එය අවම කරයි (optimisation)
Output units	Output nodes කීයද සහ කුමන output function එකද – කාර්යය මත රඳා පවතී

සෘජු ඉරක් පමණක් නොවන්නේ ඇයි? Sigmoid

Regression output එකක් සඳහා $w_1x_1 + w_2x_2 + \dots$ linear එකතුවක් හොඳයි. නමුත් output එක categorical නම් (ඔව්/නැහැ, 0/1), සෘජු ඉරක් වැඩ කරන්නේ නැහැ. එබැවින් 0 සහ 1 අතර යන **logistic වක්‍රයක්** භාවිතා කරනවා – neural-network භාෂාවෙන් මෙය **sigmoid** function එකයි.

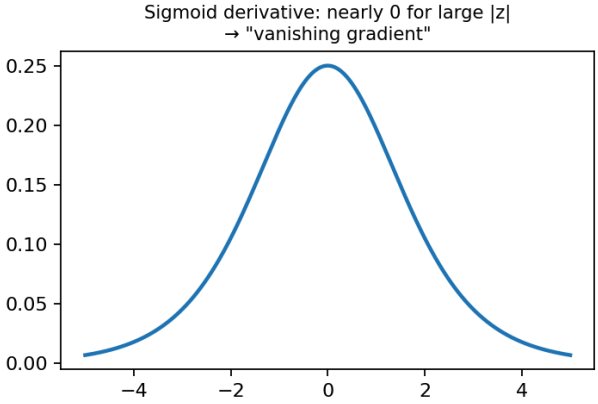
16.6 Activation functions

Activation function එකක් **non-linearity** ලබා දිය යුතු අතර training අතරතුර එහි gradient සාධාරණ ලෙස විශාලව පැවතිය යුතුයි. සුලබ තේරීම්:



Function	සූත්‍රය	සටහන්
Sigmoid (logistic)	$1 / (1 + e^{-z})$	Output (0, 1) අතර. Domain එකේ බොහෝ කොටසක derivative ≈ 0 → back-propagation අතරතුර **vanishing gradient**
tanh (hyperbolic tangent)	tanh(z)	Output (−1, 1) අතර; සමාන S-හැඩය, 0 මත කේන්ද්‍රගතයි
ReLU (Rectified Linear Unit)	max(0, z)	Origin හරහා යන linear ඒකකයක්, නමුත් "rectified": සෘණ අගයන් 0 බවට පත් වේ. වැඩිපුරම භාවිතා වන සහ දැනට වඩාත්ම සාර්ථක; tanh ට වඩා හොඳින් වැඩ කිරීමට නැඹුරුයි
Leaky ReLU	max(αz, z), කුඩා α	Derivative සෑම තැනකම ශූන්‍ය නොවේ ("dying ReLU" විසඳයි); ප්‍රතිඵල සැමවිටම ස්ථාවර නැහැ; α තෝරා ගත යුතුයි

ReLU හි විශාල වාසි දෙක: (1) $z > 0$ විට vanishing gradient නැහැ (slope එක 1); (2) **sparsity** – $z < 0$ විට output එක හරියටම 0, එබැවින් neurons බොහොමයක් "off" වේ.



Weights ඇස්තමේන්තු කරන ආකාරය (**back-propagation**) ඊළඟ වතාවේ ආවරණය කරනවා – ඒ සඳහා "vanishing gradient" මතක තබා ගන්න.

16.7 Loss function

Observations ස්වාධීන යැයි උපකල්පනය කරමු. Training මගින් **likelihood එක උපරිම කරයි** (log එක one-to-one නිසා log-likelihood එකත් එයමයි). එබැවින් loss function එක **negative log-likelihood** වේ:

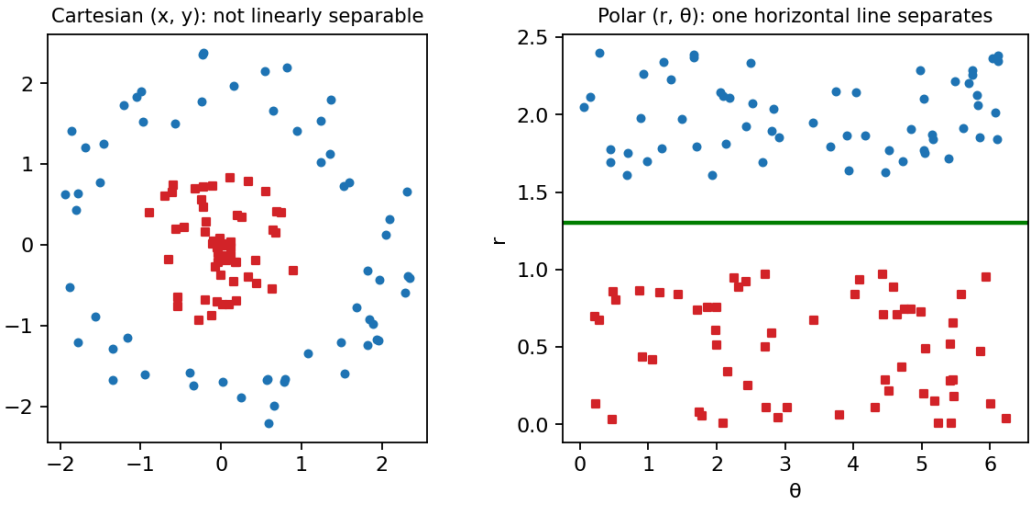
- y **Normal** නම් (continuous output) → loss එක **mean squared error (MSE)** බවට පත් වේ.
- y **Bernoulli** නම් (binary output) → loss එක **(binary) cross-entropy** බවට පත් වේ.

16.8 Output units

Output වර්ගය	Output ව්‍යාප්තිය	Output layer (activation)	Loss function	උදාහරණය
Binary	Bernoulli	Sigmoid (node 1)	Binary cross-entropy	ඔව් / නැහැ
Discrete (classes ගොඩක්)	Multinomial	Softmax (එක් class එකකට node එකක්)	Cross-entropy	iris: nodes 3 – setosa, versicolor, virginica
Continuous	Gaussian	Linear	MSE	ලකුණු, mpg
Continuous	Arbitrary	(උදා: mixture / GANs)	විවිධ	උසස්

එබැවින් output nodes ගණන වැදගත්: regression කාර්යයකට (ලකුණු පුරෝකථනය) output node **1** ක්; class 3 ක classification එකකට (iris) output nodes **3** ක්.

16.9 නිරූපණය (representation) වැදගත්



එකම දත්ත Cartesian සහ polar coordinates වලින්

(x, y) coordinates සමඟ classes (රවුම ඇතුළත vs පිටත වළල්ල) ඉරකින් බෙදිය නොහැක. **Polar** coordinates (r, θ) වලින් එක් තිරස් ඉරක් ඒවා වෙන් කරයි. හොඳ නිරූපණයක් සොයා ගැනීම යතුරයි – neural networks නිරූපණය තමන්ම **ඉගෙන ගනී**.

ප්‍රවේශය	ක්‍රියාවලිය
Rule-based systems	Input → අතින් සැලසුම් කළ program → output
සම්භාව්‍ය machine learning	Input → **අතින් සැලසුම් කළ features** → features වලින් mapping → output
Representation learning	Input → **ඉගෙන ගත් features** → mapping → output
Deep learning	Input → සරල features → වඩා ව්‍යුක්ත features (layers ගොඩක්) → mapping → output

Linear models වලින් ඔබ්බට: linear models කාර්යක්ෂමව fit වුවත් ධාරිතාව සීමිතයි. සම්භාව්‍ය ML $w^T s(x)$ භාවිතා කරයි, මෙහි s යනු **අතින් සැකසිය යුතු** non-linear පරිවර්තනයකි (domain-specific). Kernel ක්‍රම (RBF SVM වැනි) එක් ස්ථාවර තේරීමකි. **Deep learning** හි feature

පරිවර්තනය **ස්වයංක්‍රීයව ඉගෙන ගනී**: image එකක් දුන් විට network එක features තමන්ම සොයා ගනී ("embeddings" සාදයි) – එය neural networks වල සුන්දරත්වයයි.

16.10 පන්තියේ සඳහන් කළ යෙදුම්

- **අත් අකුරු හඳුනා ගැනීම:** අතින් ලිපියක් ලියා, photo එකක් ගත් විට එය Word document එකක් බවට පත් වේ. සම්භාව්‍ය dataset එක: **MNIST** (අතින් ලියූ ඉලක්කම්).
- **Image recognition:** මෙය පුද්ගලයෙක්ද, වාහනයක්ද, සතෙක්ද?
- **ස්මාර්ට් traffic lights:** camera එකක් ambulance එකක් (siren සහ lights) හඳුනාගෙන light එක කොළ පාට කරයි.
- **මුහුණු හඳුනාගැනීමෙන් පැමිණීම (attendance):** දේශකතුමාගේ කණ්ඩායම විශ්වවිද්‍යාලයේදී, දේශන ශාලාවට ඇතුළු වන සිසුන් හඳුනාගෙන attendance mark කරන model එකක් train කළා – ඉතා නිවැරදියි.
- **Layers ධුරාවලියක් ඉගෙන ගනී:** පළමු hidden layer → දාර (edges); දෙවන → කොන් සහ contours; තෙවන → වස්තුවේ කොටස්; ඉන්පසු සම්පූර්ණ වස්තුව. සමහර විට layer එකක් කරන්නේ කුමක්ද කියා පැහැදිලි කළ නොහැක – එය **black box** එකක් වගේ.
- **Transfer learning:** එක් කාර්යයක් සඳහා දැනටමත් train කළ model එකක් (උදා: ඉලක්කම්) වෙනත් කාර්යයකට (උදා: ගස් හඳුනා ගැනීම) නැවත භාවිතා කළ හැක – අපි දැනුම මාරු කරනවා.

ඊළඟ වතාවේ: neural networks පිළිබඳ exercises, back-propagation මගින් weights ඇස්තමේන්තු කිරීම. Lab sheet (සඳුදා), homework, videos සහ group-project උපදෙස් සඳහා LMS පරීක්ෂා කරන්න.

17. ප්‍රධාන කරුණු සහ R cheat-sheet

- SVM එකක් **maximum-margin** decision boundary එක තෝරා ගනී (මැද ඇති ඉර).
- **Support vectors** යනු මායිම තීරණය කරන observations වේ.
- **Soft margin** එක slack variables ξ හරහා overlap වලට ඉඩ දෙයි; **C** දඬුවම පාලනය කරයි (විශාල C → පටු margin, overfit අවදානම; කුඩා C → පළල් margin, underfit අවදානම).
- Dual problem එක inner products පමණක් භාවිතා කරයි → **kernel trick** එක පරිවර්තිත variables නොසාදා non-linear මායිම් ලබා දෙයි.
- Multi-class SVM = **one-vs-one** sub-models + majority vote (models $K(K-1)/2$).
- **ε-SVR** ε-පළල tube එක ඇතුළත errors නොසලකා හරියි; පිටත points support vectors වේ.
- දත්ත scale කරන්න, training set එක මත cross-validation මගින් tune කරන්න, සුදුසු මිනුම් භාවිතා කරන්න, test set එක ස්පර්ශ නොකරන්න.
- වඩා නම්‍යශීලී kernel එකක් **ස්වයංක්‍රීයව** වඩා හොඳ model එකක් නොවේ (mtcars උදාහරණය!).
- Neural networks: input – hidden – output layers; perceptron = බර සහිත එකතුව + bias + activation; ReLU වඩාත්ම සුලබ activation එකයි.

කාර්යය	R code (package e1071)
Classification SVM	svm(y ~ ., data = train, kernel = "linear" / "radial", cost = 1)
Regression SVR	svm(y ~ ., data = train, type = "eps-regression", kernel = "radial", epsilon = 0.1)
Default සැකසුම්	type = C-classification (factor y), kernel = radial, cost = 1, gamma = 1/(predictors ගණන), epsilon = 0.1, scale = TRUE
Predict	predict(model, newdata = test)
Confusion matrix, accuracy	table(Observed = test$y, Predicted = pred); mean(pred == test$y)
Support vectors ගණන	model$tot.nSV (print(model) මගින්ද පෙන්වයි)
5-fold CV මගින් tune	tune(svm, y ~ ., data = train, ranges = list(cost = 2^(-1:5), gamma = 2^(-4:2)), tunecontrol = tune.control(cross = 5))
හොඳම model / parameters	tuned$best.model ; tuned$best.parameters
Probabilities	svm(..., probability = TRUE); predict(m, test, probability = TRUE)
Class weights	svm(..., class.weights = c(A = 2, B = 1))
මායිම plot කිරීම (predictors 2)	plot(model, data) හෝ plot(model, data, x2 ~ x1)
RMSE / MAE	sqrt(mean((obs - pred)^2)) ; mean(abs(obs - pred))
Train/test split	ids <- sample(1:nrow(d), 0.8*nrow(d)); train <- d[ids,]; test <- d[-ids,]